以下为AI生成的文稿笔记的内容

一、基本概念 00:00

1.数据 00:03

1) 数据、数据元素、数据项的概念及关系 00:05

第一章 考点一：数据、数据元素与数据项

【核心概念】

- **数据**：能被计算机处理的**符号**（广义）
  - 狭义：数字（如1、2、3）
  - 广义：**音乐、图片、视频**等
- **数据元素**：数据的**基本单位**
- **数据项**：**不可分的最小标识单位**

【层级关系】

- **数据 ⊃ 数据元素 ⊃ 数据项**
- 构成关系：**数据由数据元素组成，数据元素由数据项组成**

【生活类比】

- 数据 = **一栋住宅楼**
- 数据元素 = **一套房子**（基本单位）
- 数据项 = **一个房间**（不可再分，如厨房）

【实例：高考成绩表】

- 整个表 = **数据**
- 每一行（如张三） = **数据元素**
- 每科成绩（如语文120分） = **数据项**
  - 注意：**120分不可再拆**（拆成12和0则失去意义）

2) 例题1:关于数据项和数据元素的叙述 02:48

数据组织层次与基本单位概念辨析

【考点定位】

- 题型：**概念辨析选择题**
- 核心范围：**数据组织的三个层次、数据元素与数据项**

【核心概念：数据组织的三个层次】

- 层次关系：**由大到小**依次排列
- 最大层次 → 中间层次 → 最小层次
- 关键：**三个层次之外的概念不属于数据组织层次**

【考点：数据基本单位】

- 基本单位：**数据元素**
- 原因：**数据元素能表达完整信息**
  - 例：**六门高考成绩合在一起** = 一个数据元素 → 表达完整高考信息
  - 单门成绩 ≠ 完整信息 → **不能作为基本单位**
- 对比：**数据项** = 最小标识单位
  - 例：**六门成绩中的一门** = 一个数据项

【考点：数据项与数据元素关系判断】

- 选项A：数据项只能是数值类型
  - 判断：**错误**
  - 依据：**广义数据** = 一切能处理的符号与信息，**不限于数值**
- 选项B：数据项可以包含数据元素
  - 判断：**错误**

- 依据：组成关系为**数据元素包含数据项**，方向相反
- 选项C：数据元素是数据基本单位
  - 判断：**正确**
- 选项D：数据元素由数据项组成的集合
  - 判断：**错误**
  - 依据：**集合内元素无关联**，而**数据项之间存在关联**，不能称为集合

【易混淆点】

- **数据元素**：基本单位，表达完整信息
- **数据项**：最小标识单位，不可再分
- **集合**：元素间无关联，不适用于有结构关系的数据项

2数据结构 04:54

数据结构的概念与分类

【结构与数据结构】

- **结构**：**元素之间的关系**，即元素的组合方式
- 类比：土木结构（钢筋水泥组合方式不同 → 高楼/平房）
- **数据结构**：**数据元素**按特定结构组合而成
  - 材料：**数据元素**
  - 结构：**组合方式**

【数据结构分类】

- **两大类**：
  - **逻辑结构**
  - **存储结构**
- 后续章节：**详细展开介绍**

1) 逻辑结构 06:36

- 逻辑结构概述 06:38
  - 逻辑结构的概念与分类
  - **【逻辑结构定义】**
    - **数据元素之间的关系**
    - 仅讨论元素间**逻辑关系**，不涉及计算机实现
    - 计算机实现方式 → **存储结构**（后续讲解）
  - **【关系示例】**
    - 陌生人之间：**无关系**
    - 师生关系：**一对多**（1个老师对应30个学生）
- 四种基本逻辑结构 07:16
  - 数据结构四种基本逻辑结构
  - **【逻辑结构总览】**
    - **四种基本结构**：集合、线性结构、树形结构、图形结构
    - 核心区分依据：**元素间关系**
  - **【集合结构】**
    - 特点：**元素之间无任何关系**
    - 示例：四个元素（abcd）分散存在，彼此孤立
    - 本质：**无关联、无次序**
  - **【线性结构】**
    - 特点：**呈现先后顺序**，形成链条
    - 关键约束：**一对一关系**
      - 每个元素向前、向后**最多仅有一个直接关联**
      - 示例：a→b→c→d，a向后唯一指向b，无分叉
    - 本质：**线性次序、单链关联**

- **【树形结构】**
  - 特点：**层次化**，逐层展开
  - 关键约束：**一对多关系**
    - 上层元素可向下关联多个下层元素
    - 示例：a向下对应b、c、d（第一层→第二层）；或a向下对应两个节点（多层结构）
  - 本质：**分层递进、一对多分支**
- **【图结构】**
  - 特点：**网状结构**，元素间可任意连接
  - 关键约束：**多对多关系**
    - 无连接限制，任意元素间可建立关联
  - 本质：**无约束、任意互联**
- **【结构复杂度对比】**
  - **复杂度排序**：图结构 > 树形结构 > 线性结构 > 集合结构
  - 原因：**限制越多，结构越简单**
    - 集合：禁止连接 → 最简单
    - 线性：仅允许前后单链 → 限制性强
    - 树形：仅允许上下层关联 → 限制较多
    - 图：无限制 → 最复杂
- **【重点分类】**
  - **线性结构**：研究最多、结构最简单
  - **非线性结构**：集合、树形、图形结构统称

- 例题1：任何两个结点之间都没有逻辑关系的逻辑结构 10:48
  - 树形结构同一层的之间，它是不能关联的
  - **【核心考点：四种基本逻辑结构】**
    - **树形结构**：**同一层节点间不能关联**
    - **四种基本逻辑结构**：**集合、线性结构、树形结构、图结构**
      - 考查方式：**任意挖空其一**，要求补全或全部写出
    - **集合结构**：**任意两个节点（元素）之间没有任何关系**
- 例题2：数据元素之间均为一对一关系的逻辑结构 11:27
  - 这两个题我们可以一起来看啊，我们说元素之间一对一的，我们是叫做线性关系吧，哎，就是线性结构嘛好，所以它就是呈现像一个链条一样的啊，好就是一个一个对一个好，然后树的特点是什么呢？它是层次的，然后每一层从上到下可一对多嘛，所以就叫一对多，再看这道题啊，
  - **【核心考点】数据结构逻辑关系**
    - **线性结构**：元素间**一对一**关系
      - 形态：**链条状**依次连接
    - **树形结构**：元素间**一对多**关系
      - 特征：**层次化**，自上而下逐层展开
- 例题4：四种基本结构中最复杂的结构 11:46
  - 四种基本结构复杂度比较
  - **【核心考点】四种基本结构复杂度排序**
    - **图**：限制最少 → **结构最复杂**
    - **其他结构**：限制较多 → 结构相对简单
    - **规律**：**限制越多，结构越简单；限制越少，结构越复杂**
    - **类比理解**：人际关系中限制多（如禁止师生恋、办公室恋情）→ 关系简单；无限制 → 关系复杂
- 例题5：逻辑结构的分类 12:17
  - 数据结构分类

- **【核心考点】数据结构四大基本结构分类**
  - **两大类划分**：
    - **线性结构**：基本、研究较多
    - **非线性结构**：其余结构统称
  - 本题答案：**B**

2) 存储结构 12:38

- 存储结构的概念与四种基本类型 12:40
  - 存储结构（物理结构）基本概念
  - **【存储结构定义】**
    - **存储结构 = 物理结构**（同义不同名）
    - 逻辑结构：**仅构想**，不涉及计算机具体实现
    - 存储结构：解决**四种逻辑结构**在计算机中的**实现与存储方式**
  - **【四种基本存储结构】**
    - **顺序存储**
    - **链式存储**
    - **索引存储**
    - **散列存储**
- 顺序存储 13:31
  - 第一种顺序存储
  - **【顺序存储】**
    - 定义：数据元素在计算机存储器中**依次相邻存放**
    - 形象比喻：存储器视为**一排存储柜**，元素**一个挨着一个**存放
    - 实现工具：**数组**
    - 核心特征：**物理位置相邻**，逻辑关系与物理关系一致
- 链式存储 14:09
  - 叫做链式存储好
  - **【链式存储】**
    - **核心思想**：存储格**不要求物理连续**（可相邻，亦可分散）
    - **连接方式**：通过**指针**串联各存储格
    - **指针作用**：类似“牵线”，**由前一结点定位后一结点**
    - **类比说明**：学生座位不必紧邻，**通过连线即可由一人找到另一人**
    - **与顺序存储区别**：顺序存储**必须紧邻**；链式存储**无此限制**
- 索引存储 15:15
  - 索引存储
  - **【考情定位】**
    - **非重点**：基本不考，了解即可
    - 重点研究对象：**顺序存储、链式存储**
  - **【核心概念：索引存储】**
    - 类比：**书的目录**
    - 作用：**锁定查找范围**，无需遍历全部数据
    - 示例：找第三章内容 → 定位在100-200页 → 排除前后无关区域
  - **【关键特征】**
    - 存储位置：**按索引规定范围存放**
    - 查找逻辑：先查索引 → 确定大致范围 → 缩小搜索空间
    - 与顺序/链式存储的区别：**非直接定位，仅范围锁定**
- 散列存储 16:06
  - 散列存储（哈希存储）概念
  - **【存储结构：散列存储（哈希存储）】**

        - **定义**：又称**哈希存储**，二者同义
        - **本质**：**函数映射**（如 f(x)=y）
            - 输入：待存数据
            - 输出：**存储地址**
        - **核心函数**：**哈希函数**（记作 **h**）
            - 示例：数据2代入 → 结果0 → 存于0号位置；数据5代入 → 结果2 → 存于2号位置
        - **学习定位**：后续章节**详细介绍**，当前阶段**建立概览性认知**即可
- 例题1:数组的存储结构 17:06
    - 数组采用顺序存储
    - **【核心考点】**
        - **数组**：**顺序存储**结构
        - 定位：**四种存储结构**之一（顺序存储）
- 例题2:四种存储结构判断 17:18
    - 他说不属于四种常。常用的存储存储方式就是存储结构嘛，就四种结构好，那么顺，我们是顺序链式，然后我们是。呃，这个散呃索引和这个散列对吧？那么没有这个啊，这个应该是索引，所以这个选c再来看一下这道题啊
    - **【考点：存储结构分类辨析】**
        - **四种基本存储结构**：**顺序存储、链式存储、索引存储、散列存储**
        - 题干要求：选出**不属于**四种常用存储结构者
        - 选项分析：
            - 顺序、链式：**属于**
            - 索引、散列：**属于**
            - 干扰项：**非索引/非散列**（如“直接存储”等）→ **不属于**
        - 结论：**正确选项为C**（对应非四种结构之项）
- 例题3:链式存储的存储空间判断 17:34
    - 判断题
    - **【考点：链式存储空间连续性判断】**
        - **结论**：**错误**
        - **核心辨析**：
            - 链式存储**不要求**物理空间连续
            - ≠ 一定不连续
            - 物理空间**可连续，也可不连续**
        - **关键点**：**“不要求连续” ≠ “一定不连续”**
    - 第一章的第五个考点抽象数据类型
    - **【考点：抽象数据类型（ADT）】**
        - **定义**：**抽象数据类型**（Abstract Data Type）
        - **核心特征**：
            - **数据抽象**：仅关注**逻辑特性**，屏蔽具体实现
            - **信息隐藏**：操作的具体细节对外不可见
        - **三要素**：
            - **数据对象**
            - **数据关系**
            - **基本操作**
        - **重要区分**：
            - ADT ≠ 具体数据结构
            - 与**物理存储方式无关**

3) 抽象数据类型 18:14

- 类型与运算的基本概念 18:18
  - 类型与运算的基本概念
  - **【核心概念：类型】**
    - **定义**：**一类值的集合**
      - 例：整数类型 = {1, 2, 3, ...} 的集合
    - **通俗理解**：**分类**
      - 例：人类分为男人、女人
      - 本质：**将具有相同特征的对象归为一类**
  - **【核心概念：运算】**
    - **定义**：**操作**
    - **与类型的关系**：**类型与运算紧密关联**
      - 特定类型对应特定运算
      - 例：整数类型 → 可进行**加减乘除**运算
    - **核心逻辑**：**确定类型即确定可执行的运算集合**
- 原子类型与结构类型 19:21
  - 原子类型与结构类型的概念
  - **【核心概念】**
    - **原子类型**：**不可再分**的数据类型
      - 例：**整型**（整数）
    - **结构类型（结构体）**：**可再分**的数据类型
      - 构成：由**多个原子类型**组合而成
    - 关系：**结构类型由原子类型构成**
- 抽象数据类型（ADT）的定义与特点 19:42
  - 抽象数据类型（ADT）的概念
  - **【核心定义】**
    - **抽象数据类型（ADT）**：**自行定义、构想或创造的类型**
    - 核心特征：**定义类型及其可执行操作，不涉及具体存储与实现**
  - **【通俗类比】**
    - 自定义类型示例：**“蔬菜”类型**
      - 包含值：**萝卜、白菜**等
    - 自定义操作示例：**煎、炒、烹、炸**
    - 本质要求：**自圆其说即可，仅需构想，无需计算机内具体实施**
- 例题1：定义抽象数据类型时不需要做的事情 20:38
  - 抽象数据类型定义与算法考点
  - **【考点：抽象数据类型（ADT）】**
    - **核心原则**：**无需具体实现**，仅需描述逻辑特性
    - **ADT三要素**：
      - **类型名称**：如“蔬菜”（Vegetable）
      - **类型操作**：如“煎、炒、烹、炸”四个操作
      - **元素描述**：说明类型划分及操作规则
    - **题目解析**：
      - 选项C：**“不需要具体实现”** → 正确
      - 选项A：给出类型名字 → 符合ADT定义
      - 选项B：定义类型上的操作 → 符合ADT定义
      - 选项D：用某种元素描述抽象数据类型 → 符合ADT定义
  - **【考点：算法（第一章第六个考点）】**
    - **定位**：**第一章第六个考点**
    - **核心**：算法相关概念与特性

4) 算法 21:29

算法的概念与特性

【核心概念】

- **算法定义**：**解决问题的步骤**
- 实例：上海→北京
  - 方案一：出租车→机场→飞机
  - 方案二：出租车→火车站→火车
- 结论：**同一问题可有多重算法**

【算法五大特性】（考点：填空/大题）- 输入：可有**零个或多个 - 输出**：**至少一个或多个 - 有穷性**：**步骤必须有限**，不可无限运行

- **确定性**：**每一步无歧义 - 可行性**：**操作必须能实际执行**
- 反例："把大象装进冰箱"三步法——**缺乏可行性**

算法的评估标准

【评估必要性】

- 同一问题多算法 → 存在**优劣之分**
- 评估标准：**时间复杂度**、**空间复杂度**

【时间复杂度】

- 定义：**算法执行语句的总数**
- 表示法：**大O表示法**
  - 规则一：**只取执行语句的主体**（增长最快项）
  - 规则二：**忽略系数与低阶项**
  - 示例：执行总数 = $\frac{1}{2}n^2 + n + 3$ → **$O(n^2)$**
- 常见大O表示法（**效率从高到低**）：
  - **O(1)**：常数级（无论常数多大，如1万、100万）
  - **O(n)**：线性级（如$\frac{1}{2}n+5$ → O(n)）
  - 其余依主体类推
- 效率排序：**O(1) 最高，依次递减**

【空间复杂度】

- 定义：**算法运行过程中临时占用的空间**
- 考频：**研究较少**
- 例题:算法时间复杂度 25:27
  - 算法与时间复杂度概念题解析
  - **【考点：算法定义】**
    - **算法**：**求解问题的步骤**
    - 题型：概念题
    - 正确选项：D
  - **【考点：算法评估指标】**
    - 不同算法存在**好坏之分**，需评估
    - 评估指标：**时间复杂度**、**空间复杂度**
    - 题型：概念题
    - 正确选项：B
  - **【考点：时间复杂度本质】**
    - 时间复杂度：**语句执行次数**（计算量）
    - 依赖**给定输入n**（n确定后计算量固定）
    - 示例：n=10时，语句数=100
    - 题型：概念题
    - 正确选项：C
  - **【易混淆点：为何不用执行时间】**

        - 原因：**不同软硬件环境**下执行时间差异大，不具备可比性
        - 故以**语句频度**衡量时间复杂度
- 例题:时间复杂度 26:15
    - 时间复杂度基本概念
    - **【核心考点：时间复杂度与运行效率】**
        - **运行效率差异**：同一程序在不同性能电脑上运行时间不同
        - **关键区分**：**运行时间 ≠ 算法效率**
        - **时间复杂度本质**：衡量**算法执行效率**，与硬件性能无关
    - **【考点：常数阶 O(1)】**
        - **表示形式**：**O(1)**
        - **易混淆点**：**O(1) ≠ 只有一条语句**
        - **正确理解**：语句数量为**常数条**（如3句、100句）均表示为 **O(1)**
- 例题:大O表示法阶数 26:41
    - 大O表示法阶数比较
    - **【核心考点】**
        - **大O表示法**：用于衡量算法复杂度，**阶数越小**，语句执行次数越少
        - 本题实质：**给定n值，比较各复杂度表达式大小**
    - **【解题思路】**
        - 方法：**设n=10**，代入各表达式求值
        - 各表达式计算结果：
            - **$\log_2 n$**：$\log_2 10 \approx$ **- 几**（介于3与4之间）
            - **n**：**= 10**
            - **n^2**：**= 100**
            - **2^n**：**= 1024**
        - 结论：**$\log_2 n$（选项A）阶数最小**
- 例题:顺序表时间复杂度 27:20
    - 时间复杂度分析
    - **【核心考点】**
        - **大O表示法**：只抓**主体**，忽略多一句少一句
        - **主体识别**：找**循环**（while/for）
        - **循环分析**：
            - 条件：i < n
            - 初始值：i = 0
            - 操作：i 每次加1
            - 执行次数：**约n次**（n-1、n、n+1均可视为n）
        - **结论**：时间复杂度为 **O(n)**，选A
    - 线性表的定义
    - **【核心概念】**
        - **线性表**：**线性结构**的数据元素序列
        - **数据元素**：数据的基本单位
        - **逻辑结构**：元素间**一对一**、一个接一个
        - **存储结构**：**待定**（未指定）
    - **【存储结构分类】**
        - **顺序存储 → 顺序表**
        - **链式存储 → 链表**
    - **【位置概念】**
        - **位置** = 元素**下标**
        - **下标从0开始**

        - 第1个元素 → 位置0（a0）
        - 第2个元素 → 位置1（a1）
        - 第3个元素 → 位置2（a2）
        - 第4个元素 → 位置3（a3）
    - 易混淆：**位置2** 表示第**3**个元素
- 线性表性质判断题
- **【题目1：线性结构性质】**
    - **线性结构**：节点数不为0
    - **首节点**：**无前驱**
    - **其他节点**：**仅一个直接前驱**
    - 关系：**一对一**
- **【题目2：选错误项】**
    - A：**正确** — 除首元素外，每个元素**仅一个前驱**
    - B：**正确** — 除末元素外，每个元素**唯一后继**
    - C：**正确** — 元素个数可为0，即**空表**
    - D：**错误** — 线性表为**有限序列**，元素个数**必须有限**
- **【题目3：选正确项】**
    - A：**错误** — “离散”表示元素间**无关联**，线性表元素**有关联**
    - B：**错误** — 首元素**无前驱**，并非“有且仅有”
    - C：**正确** — 可取元素、查找、排序，均为**基本操作**
    - D：**错误** — 可在**任意位置**插入和删除元素
- 顺序表
- **【核心概念】**
    - **顺序表** = 线性表 + **顺序存储**
    - 存储形态：**连续格子**，一个挨着一个
    - 实现工具：**数组**
- **【数组要点】**
    - 表示：a[n]，n为**容量**
    - 下标范围：**0 ~ n-1**（非1~n）
    - 例：a[10] → 下标 **a0~a9**
- **【顺序表特性】**
    - **预留足够空间**：过少放不下，过多浪费
    - **内存连续**：物理地址连续（0,1,2, - ..）
    - **相邻元素地址相邻**：逻辑相邻 = 物理相邻
    - **下标从0开始**
    - **随机访问方式**：
        - 直接计算地址并访问
        - 依据：地址有规律，可**直接偏移计算**
- **【题目1】**
    - 线性表 + 顺序存储 → **顺序表**
    - 实现工具：**数组** → 选C
- **【题目2：地址计算】**
    - 20个元素存入数组，容量≥20
    - 第9个元素地址：1000
    - 第11个元素地址：1040
    - 间隔：2个元素 → 地址差40 → **每元素占20**
    - 第20个元素与第11个元素间隔：**9个元素**
    - 计算：1040 + 20×9 = **1040 + 180**

- 结论：选对应选项
- 技巧：**不数格子**，按**地址规律计算**

3线性表 36:55

比如说比如说这个第一个格子和第二个格子相隔几个，就只它的，它就是相隔一个嘛，所以你就会发现你只需要用它的这个第几个相减就行了啊，所以第20个和第11个它就是相隔20-11就是相。隔九个啊，也就相隔九个，所以这个九个这个九呢，你不用数，你就直接用20-11，然后再加上这个。加上这个地址就可以了。

【核心方法：间隔数计算】

- **间隔数**：**第n个与第m个之间相隔 (n - m) 个格子**
- **示例**：第20个与第11个 → **相隔 20 - 11 = 9 个**
- **操作要点**：**无需逐个数**，直接**序号相减**即可
- **后续步骤**：用**相隔数 + 起始地址** 定位目标

1) 顺序表的插入 37:18

第二章的第三个考点顺序表的插入

【核心操作逻辑】

- **顺序表存储特性**：物理存储**连续无空位**，中间不可直接插入
- **插入操作步骤**：
  - 从**插入位置起**，后续元素**依次后移一位 -** 空出目标位置
  - 填入新元素
- **本质**：**从插入位置开始，后面全部往后移动**

【考点：平均移动次数推导】

- 场景：长度为 **n** 的顺序表，任意位置插入
- 可插入位置数：**n+1**（含表尾后）
- 各位置移动次数：
  - 插在**第1个元素前**：移动 **n** 次
  - 插在**第2个元素前**：移动 **n-1** 次
  - ……
  - 插在**第n个元素前**：移动 **1** 次
  - 插在**表尾后**：移动 **0** 次
- 平均移动次数计算：
  - 总移动次数 = **1 + 2 + ... + n = n(n+1)/2**
  - 平均 = **n(n+1)/2 ÷ (n+1) = n/2**
- **结论（必记）**：顺序表任意位置插入，**平均移动次数 = n/2**

【典型例题解析】

- 例1：9个元素，在**第3个元素前插入**
  - 实际插入位置：**第3个元素处**
  - 需移动元素：**3~9**，共 **7** 个
  - 答案：**C**
- 例题:顺序表插入时间复杂度 40:55
  - 第二章考点四：顺序表插入操作时间复杂度分析
  - **【核心结论】**
    - **顺序表任意位置插入**：平均移动元素次数为 **n/2**
    - 推导过程：已在前文完成，此处直接引用结论
    - 考试应急：若遗忘结论，可现场推导
  - **【题目解析】**
    - 题干：在任意位置插入元素，执行移动语句次数
    - 计算：**移动次数 = n/2**

        - 大O表示法：**忽略系数**，n/2 → **O(n)**
        - 答案：**C**

2) 顺序表的删除 41:25

- 顺序表删除操作及平均移动次数计算 41:29
    - 顺序表的删除操作
    - **【核心考点：顺序表删除操作】**
        - **删除本质**：**减少元素**，与插入（增加元素）相对
        - **核心原则**：删除后**必须保持元素连续相邻**，不允许出现空位
    - **【删除操作步骤】**
        - 以删除元素“3”为例（原表：1,2,3,4,5）
            - **不可直接移除**：直接移除会导致空位，破坏顺序表结构
            - **正确做法**：将**被删元素之后的所有元素依次前移一位**
                - 4 → 原3位置
                - 5 → 原4位置
            - 结果：元素“3”被覆盖，表仍保持连续（1,2,4,5）
    - **【考点：平均移动次数推导】**
        - 问题：长度为n的顺序表，删除任意位置元素，**平均移动次数**
        - 分情况分析（删除第i个元素）：
            - 删第1个：需移动 **n-1 次**（2至n全部前移）
            - 删第2个：需移动 **n-2 次**（3至n前移）
            - …
            - 删第n个：需移动 **0 次**（无后继元素）
        - 移动次数序列：**n-1, n-2, …, 0**（等差数列）
        - 平均次数计算：
            - 总和 = (n-1 + 0) × n / 2
            - 平均 = 总和 / n = **(n-1)/2**
        - **结论（直接记忆）**：顺序表删除操作**平均移动 (n-1)/2 次**
- 例题1:长度为n的顺序表中删除一个元素需移动元素的平均次数 43:44
    - 来看一下题目啊，这呢就是直接考结论就是长度为n中删除一个，他这时候删除一个就是不知道哪一个就任意删除啊，就是所以它的平均次数，这就直接是结论二分之n减一嘛。当然，如果说你考试时候忘了，
    - **【考点：顺序表删除操作平均移动次数】**
        - **直接结论**：长度为 **n** 的线性表，**任意位置删除**一个元素
            - 平均移动次数 = **（n-1）/ 2**
        - **易忘应对**：若考试遗忘，可**现场推导**（基于等概率假设，各位置删除概率均为 **1/n**，移动次数分别为 **n-1, n-2, ⋯, 0**，求和取平均）
- 单链表的概念及节点构成 43:58
    - 第二章第五个考点：单链表
    - **【考点定位】**
        - **单链表**：线性表链式存储结构
        - 逻辑结构：线性表（仅定逻辑，未定存储）
        - 存储结构：顺序存储→顺序表；链式存储→链表
    - **【核心概念：节点结构】**
        - 节点构成：**数据域 + 指针域**
        - 数据域：存储元素值（如a、b、c）
        - 指针域：**指向下一节点**，实现节点间连接
    - **【节点连接逻辑】**
        - 首节点a：指针域→节点b
        - 节点b：指针域→节点c

        - 末节点c：**指针域为空**（图示符号表示无后继）
        - 多个节点依次链接→形成**单链表**

3) 单链表 45:15

单链表基础概念与结构

【单链表构成】

- **节点**：ABC三个节点依次相连，形成**单链表**
- **头节点**：链表前设置**无实际数据**的节点，起**标志作用**
- **头指针**：指向头节点的指针，常用 head 表示，**名称可自定义**

【节点术语区分】

- **头节点**：链表首部，不存实际元素
- **首节点**：头节点后的**第一个实际数据节点**
- **尾节点**：链表**最后一个节点**

【常用表示符号】

- ∧：表示**空值**，尾节点指针域为空
- head：**头指针**
- p->data：指针 p 指向节点的**数据域值**（如 p 指向B，则 p->data = B）
- p->next：指针 p 指向节点的**指针域**，指向下一节点

【空链表判定】

- **带头结点单链表为空**：头节点的指针域为空，即 head->next = NULL

【C语言易错点】

- =：**赋值**（如 a=5 表示将5赋给a）
- ==：**判断相等**（如 a==5 表示判断a的值是否等于5）
- 例题:单链表存储方式 47:46
    - 单链表基础概念与判断题解析
    - **【题型定位】**
        - 题目要求：选**错误**项（三对一错）
    - **【选项辨析】**
        - **A项**：节点通过**指针域**连接 → **正确**
        - **B项**：头指针变量**命名不强制**为head（可自定义或简写） → **正确**
        - **C项**：尾节点指针域为**空（NULL）** → **正确**
        - **D项**：用**尾指针**表示单链表 → **错误**
            - 正确表述：用**头指针**（指向头节点的指针）表示单链表
    - **【考点：链表存储地址连续性】**
        - **顺序表**：物理地址**必须连续**
        - **链表**：物理地址**不强制连续**（通过指针连接）
            - 注意：**可以连续，也可以不连续**（不排斥连续存放）
            - 结论：选**D**
    - **【考点：尾节点判定】**
        - 条件：指针p指向**尾节点**
        - 特征：尾节点**指针域为空**
        - 判断语句：**p->next == NULL**
    - **【考点：头结点与首节点】**
        - **头结点**：位于链表首部，**不存实际元素**，仅为操作方便
        - **首节点**：头结点之后，**第一个存有实际元素**的节点
        - 头结点的指针域指向：**首节点**
    - **【考点：空链表判定】**

        - 带头节点单链表为空条件：**头节点后无任何节点**
        - 判断语句：**head->next == NULL**
    - **【考点定位】**
        - 所属章节：**第二章第六考点：单链表的插入**
- 单链表的插入 50:15
    - 单链表的插入操作（含指针指向与顺序问题）
    - **【核心操作：两步连接】**
        - 目标：将指针 **s** 指向的节点插入至 **p** 与 **q** 之间
        - 插入后逻辑关系：**p → s → q**
        - 两步操作：
        - **s 的 next 指向 q**（连接后继）
        - **p 的 next 指向 s**（连接前驱）
    - **【关键考点：操作顺序的可行性】**
        - **情况一：p 与 q 均有独立指针**
            - 两步顺序**可颠倒**（先连后继或先连前驱均可）
        - **情况二：q 无独立指针（仅由 p->next 表示）**
            - **必须严格先执行步骤一，再执行步骤二**
            - 原因：若先执行步骤二（p->next = s），则 **q 节点失联**，原后继节点无法再被访问
    - **【易错警示】**
        - 无独立指针时，**颠倒顺序将导致链表断裂**，丢失原后继节点
    - 例题:单链表插入操作 52:25
        - 链表插入操作指针顺序辨析
        - **【核心考点：单链表插入操作指针顺序】**
            - **关键前提**：明确插入方向（**q在p前** vs **p在q前**）
            - **判断依据**：是否存在**独立指针**指向待插入节点
        - **【情形一：q在p前插入s（有独立指针）】**
            - 操作步骤：
                - **q→next = s**（q的后面是s）
                - **s→next = p**（s的后面是p）
            - **顺序可交换**：因s有独立指针，两步先后**无影响**
            - 正确选项：**C**
        - **【情形二：p后插入q（无独立指针）】**
            - 操作步骤：
                - **q→next = p→next**（q的后面是p的后继）
                - **p→next = q**（p的后面是q）
            - **顺序不可交换**：因p→next无独立指针，**必须先保存后继再连接**
            - 正确选项：**A**
- 单链表的删除 53:46
    - 第二章第七个考点 单链表的删除
    - **【核心操作】**
        - 目标：**删除单链表中指定节点**（如节点b）
        - 本质：**修改前驱节点指针域**，跨越被删节点，直接指向后继节点
        - 形象类比：队伍中**前、后小朋友直接牵手**，中间者自然出列
    - **【删除步骤】**
        - **逻辑断开**：将节点a的指针域**直接指向节点c**（跨过b）
        - **释放空间**：被删节点b仍占用存储空间，需**调用 free(p)** 释放内存
            - 注意：**第二步常可省略**，但必须知晓其存在
    - **【易错提醒】**

        - 删除操作核心是**指针重连**，而非物理清除
        - **未释放节点空间**会导致内存泄漏
    - 例题:单链表删除节点方式 55:07
        - 我们来看一下这道题啊，他说指针p指向结点a，这个是a啊，好p指向它。然后呢？它后面还有若干个啊，当然他没说一定有，就说就说现在当然要讨论，就当他有呗，好他现在说要把这个a后面的这个节点删除掉，实际上就是要把这个节点删除掉啊。那么，这个节点删除掉，我们刚才说了，我们的做法是什么？直接跨过它，把在后面这个直接连起来嘛，所以我们的呃操作实际上就是a和这个给它直接连起来，那么呢，这个连起来这个语句怎么表示你看啊？呃，我们假设假设这个是b，这个是c，那原来这个c是怎么表示啊？原来这个b是不是就是p的后面，所以是b是p的next对吧？那么c就是p的后面是是这个p的后面的后面，所以这个这个c是。p的next。然后再next。好，那么现在a和它连上了，那就变成什么a的，后面就是它对吧，所以就变成了p的，后面就等于它，所以就是p的，后面就是等于它。呃呃，那么这个就选黑了好，再看这个题啊，这个就是我们说的这个删除的节点，
        - **【题目情境】**
            - 指针 **p 指向结点 a**
            - 目标：**删除 a 后面的结点**（即结点 b）
        - **【删除操作核心思想】**
            - 做法：**直接跨过待删结点**，将 a 与后继结点 c 直接相连
        - **【指针关系推导】**
            - **b 的表示**：p->next
            - **c 的表示**：p->next->next
            - **删除后连接**：p->next = p->next->next
        - **【结论】**
            - 该语句即 **删除结点 b 的核心操作**
- 例题:单链表节点释放语句 56:09
    - 第二章第八个考点：循环链表
    - **【考点定位】**
        - 主题：**循环链表**
        - 关联操作：**空间释放（free）**
    - **【核心辨析】**
        - **free**：**释放节点空间**（本题正确选项）
        - **创建节点**：分配空间大小
        - **复制为空**：将链表置空
    - **【易混淆点】**
        - **free ≠ 创建 ≠ 置空**：三者操作对象与语义不同，需区分记忆。

4) 循环链表 56:24

循环链表的定义与特性

【循环链表定义】

- **本质**：单链表基础上的改进
- **核心改动**：尾节点指针域不再置空，改为**指向头节点**，形成循环结构

【核心优势】

- **任意节点出发可遍历全表**
- 对比单链表：单链表指针域仅单向，尾节点后为空，无法回溯
- 示例：指针p指向某节点，单链表下无法找到节点a；循环链表下，p→后继→头节点→a，**可达任意节点**

【空循环链表表示】

- 条件：头节点后无其他节点
- 形态：头节点即尾节点，指针域**指向自身**
- 判定条件：**头节点.next == 头节点自身**
- 例题:循环链表判断 58:17
  - 带头结点循环链表的判断条件
  - **【考点：带头结点循环链表判空】**
    - 判空条件：**头结点next指向自身**（头next = 头）
    - 选项：**C**
  - **【考点：指向表尾结点指针p的条件】**
    - 条件：**p->next = 头结点**
    - 原因：循环链表尾结点指针域**回指头结点**
  - **【考点：带头结点单向循环链表性质判断】**
    - 正确选项：**A**
      - 依据：**尾结点指针域指向头结点**
    - 错误项分析：
      - **B**：长度≠数据元素个数+1，**头结点不计入长度**
      - **C**：环形结构**可计算长度**，长度为实际数据结点个数
      - **D**：尾结点指针域应指向**头结点**，而非**首数据结点**（因带头结点）
  - **【章节定位】**
    - 考点：**第二章第九考点：双向链表**（前置铺垫）

5) 双向链表 59:53

双向链表的基本概念与结构

【双向链表定义】

- **节点结构**：三个域
  - **前驱指针域**（指向前一个节点）
  - **数据域**（存储元素值）
  - **后继指针域**（指向后一个节点）
- 与单链表对比：单链表仅含**数据域+后继指针域**，无法向前查找

【核心优势】

- **双向查找**：可直接访问前驱节点，无需遍历或绕环
- 操作效率提升：**前驱查找时间复杂度降为O(1)**

【双向链表示意图解】

- 带头节点结构：头节点、节点a、节点b
- 指针连接：
  - 头节点后继→a，前驱→空
  - a后继→b，前驱→头节点
  - b后继→空，前驱→a

【双向循环链表】

- 定义：利用头尾空指针，形成闭环
  - 头节点前驱→**尾节点**
  - 尾节点后继→**头节点**
- **空表判定条件**：
  - 仅剩头节点
  - 头节点前驱与后继均**指向自身**（head->next head 且 head->prior head）
- 例题:双向链表对称性表示 01:02:35
  - 双向循环链表节点结构
  - **【节点结构】**

        - **双向循环链表节点**：含**三个域**
            - **数据域**：存储数据
            - **前驱指针域**：指向**前驱节点**
            - **后继指针域**：指向**后继节点**
        - 教材改版后：**现行表示方式**为数据域居中，前驱指针居左，后继指针居右
    - 双向循环链表头结点指针指向
    - **【头结点指针指向】**
        - **非循环双向链表**：头结点前驱指针指向**空（NULL）**
        - **双向循环链表**：头结点前驱指针指向**最后一个节点**
    - 双向循环链表的对称性
    - **【对称性定义】**
        - 核心性质：**节点左右对称**
        - 具体表现（设指针p指向某节点）：
            - **p的前驱节点的后继 = p**
            - **p的后继节点的前驱 = p**
        - 数学表达：**p = p→prior→next = p→next→prior**
    - 第二章第十考点——顺序表与链表的比较
    - **【考情定位】**
        - 考点：**顺序表与链表的比较**
        - 考查方式：**常结合对比分析**
    - **【比较维度】**
        - **存储方式**：顺序表为**连续存储**，链表为**离散存储**
        - **空间分配**：顺序表**静态分配**（容量固定），链表**动态分配**（灵活扩展）
        - **存取方式**：顺序表**随机存取**，链表**顺序存取**
        - **插入/删除操作**：顺序表需**移动大量元素**（时间复杂度O(n)），链表仅需**修改指针**（时间复杂度O(1)）
        - **空间利用率**：顺序表无额外指针开销，链表需**额外存储指针域**
        - **适用场景**：顺序表适合**频繁查找**，链表适合**频繁插入删除**

6) 顺序表和链表的比较 01:04:00

- 顺序表和链表的基本概念 01:04:02
    - 顺序表与链表的对比
    - **【核心对比：顺序表 vs 链表】**
    - **| 对比维度 | 顺序表 | 链表 |**
    - |---|---|---|
    - **| 逻辑结构 | 线性表**（逻辑结构相同）**| 线性表**（逻辑结构相同）**|**
    - **| 存储方式 | 顺序存储**（数组实现）**| 链式存储**（节点相连）**|**
    - **| 物理地址 | 必须连续**（格子相邻）**| 不强制连续**（可连续可不连续）**|**
    - **| 插入/删除操作 | 不方便**（需大量移动元素腾位或覆盖）**| 方便**（仅需修改指针指向，前后连接或跨过）**|**
    - **| 特殊情形 |** 尾插/尾删较方便 | 无特殊限制 |
    - **| 空间分配 | 需预先分配**（如预估10个，预留20个空间）**| 无需预留**（来一个节点分配一个）**|**
- 例题1:关于线性表的叙述中错误的是 01:05:33
    - 我们来看一下这题目啊，这个题。关于线性表，它是选错误的，是意思就是说有三个对的，一个错的，那么我们看一下这个a和a和c可以一起拿着一起来看啊a和c。他说，线性表用链式存储，那就是链表呃，那么链表呢？它是方便插入删除的，所以这个是对的啊，然后呢？链表不用占用连续的存储空间，这个也是对的，所以这两个都是对的啊，这个题是选错的，所以在另外两个再看啊，再看b

和c一起来看。他说，线性表用顺序存储，那就是顺序表，那么顺序表是不方便进行插入删除的吧，所以这个错了，

- **【题目要求】**
  - **题型**：选择题（选**错误**项）
  - **逻辑**：三对一错
- **【选项A：链式存储】**
  - **结论**：**正确**
  - 理由：链表**方便插入删除**
  - 理由：链表**无需连续存储空间**
- **【选项C：链式存储】**
  - **结论**：**正确**
  - 理由：同A，属**链式存储优势**
- **【选项B：顺序存储】**
  - **结论**：**错误**
  - 理由：顺序表**不方便插入删除**

- 例题2:线性表L的常用操作及存储结构选择 01:06:08
  - 线性表操作与存储结构选择
  - **【题目解析：访问任意位置 + 末尾插入删除】**
    - **访问任意位置**：
      - **顺序表**：**随机访问**，直接计算地址，最优
      - 链表：需从头节点逐个查找
    - **插入/删除限制条件**：**仅在最后进行**
      - 顺序表常规劣势：任意位置插入删除需**大量移动元素**
      - 末尾操作特例：插入/删除最后元素，**无需移动**，规避劣势
    - **结论**：综合两者，选 **A（顺序表）**
- 静态链表的概念 01:07:16
  - 第二章的第11个考点静态链表
  - **【考情与定位】**
    - **新增内容**（改版后）
    - 要求：**了解基础概念**即可
    - 考法：**不会过深**
  - **【核心概念】**
    - 链表存储介质：**可用数组实现**（不限于单链表）
    - 适用类型：**单链表、单循环、双循环等**
  - **【静态链表存储原理】**
    - 数组下标：**从0开始**，即地址
    - 单链表存储示例：
      - **头结点**：数据域留空，游标指向首元素地址
      - **元素节点**：数据域存元素，游标存下一节点地址
      - **尾节点**：游标存 **-1**（表示空）
    - 头结点数据域复用：**存元素节点个数**
  - **【变式：不同链表结构的数组表示】**
    - **单循环链表**：尾节点游标由 -1 改为 **头结点地址（0）**
    - **双向链表**：需**三列**（数据域+前驱指针+后继指针），较复杂，**一般不考**
  - **【备考提示】**
    - 重点理解：**静态链表如何用数组模拟指针**
- 例题3:含6个元素的线性表保存在数组中表示的链表类型 01:09:49
  - 静态链表与链表还原题目解析
  - **【考点定位】**

- 核心概念：**静态链表**（链表以数组形式存储）
- 题目要求：**根据数组还原链表结构**并判定类型

- **【解题步骤：链表还原】**
  - **头节点**：下标0，存储值6（表示**后续共6个节点**）
  - 节点1：下标0 → 指向3 → 对应**103**
  - 节点2：103 → 指向4 → 对应**501**
  - 节点3：501 → 指向2 → 对应**205**
  - 节点4：205 → 指向1 → 对应**618**
  - 节点5：618 → 指向5 → 对应**781**
  - 节点6：781 → 指向6 → 对应**910**
  - 节点7：910 → 指向0 → **回到头节点**
- **【结论】**
  - 还原结果：**单向循环链表** → 选 **C**
- **【排除法技巧】**
  - **排除B、D（双向链表）**：双向链表需**三列**（前驱+后继），此处仅两列
  - **排除A（单链表）**：单链表末尾指针应为**-1**，此处无-1
  - **判定循环**：末尾指向**头节点下标0**，故为**循环链表**

二、栈和队列 **01:12:04**

第三章的第一个考点，咱们介绍一个新的概念，叫做占好什么是占呢？占本质上它是一个线性表，只不过呢，它对线性表做了一些限制啊，咱们举个例子，比如说咱们看这边这边有一个竖着的。的线性表啊，这个线性表我们竖着横着都行呃，如果说我没有对它做限制的话，这个线性表呢，我想在任意位置插入或者删除都可以啊，那么呢，现在我们。给它做一些限制啊，就是我们规定它只能在一端进行插入和删除，这不是有上下两端吗？我们就规定它在上面这一端进行插入和删除，其他的任意位置都不行啊，也就是说假设说我现在要。增加一个，我只能增加在这里好，假设说我现在要删除一个，我只能删除这一个，其他的都不能动啊，那么我当对这个线性表进行了这样一种限制之后，它就变成了一个栈。那么呢，就是这两端可以进行操作的，这一端也就是可以这个插入和删除，我们把这个插入叫做入站。删除叫出站啊，也就是可以进出的这一端，我们就把它叫站顶，所以这边就是站顶，然后另外一端呢，这一端我们就叫做站底，然后这个站呢，我们最形象的。方法呢，是把它理解成一个有底的一个桶，你看我们这个这边这个蓝色的框，这就是一个有底的桶，然后呢，这里面有若干个格子啊，它就是里面的元素，然后呢，你看啊，比如说我们要。呃，假设说这里面现在有AB有ABC好，假设我现在第一要进来，进来就是我们说的入站吧，它因为这下面这个桶这下面封死了，对不对？所以从下面进不来啊，所以它只能从这上面进来。他只能从这上面进来，进来之后就变成这样了好，然后呢，这个如果说我们想要删除的话，我们想删除这个ABC也删除不了，为什么呢？因为他他被这个压住了，压在下面了，他要。删除的话，只能删除这个d，也就是d从这个上面出去啊，所以这就刚好这个有底的桶就和我们这个站的这个规定是完全符合的，然后呢，站的特点用四个字总结叫做先进后出，当然还有一种，另外一种说法叫后进先出。实际上，这两这两句是一个意思啊，什么叫做先进后出呢？就比如说我们刚才假设说这个里面占里面有ababcd，这个a是最先进来对吧？a最先进来，但是他因为他压在最下面，所以他想出去的话，他必须等上面都出去了，他才能够出去，所以a是先进来的，后出去那什么叫后进先出呢？你看这个d。d是最后进来的吧，最后进来，但是它在最上面，所以要出去的时候，它必须先出去，对吧？所以这叫后进先出，那么呢，我们真实生活中的例子啊，比如说。呃，一一摞盘子或者一摞书，它实际上都是一个栈，你看你把一摞盘子摞在一起，哎，我们假设说就是一次只能取一个的话啊，那么你这个盘子在最底

下，这个你是拿不出来的，你必须得把上面的盘子全部拿走之后才能把它取出来。上面最上面那个盘子是最后放上去的，你第一个拿就是把第一个盘子拿走哎，所以这个拿走第一个盘子，这个就相当于是出站，他只能在最上面一个操作嘛，然后呢，咱们注意这样一个问题啊，相同的入站序列可以得到不不同的出站序列。为什么呢？因为你入站的同时啊，它可以一边出站啊，好，咱们举个例子，咱举个最简单例子啊，咱们就打两个元素吧a和b两个元素好，如果说我现在告诉你入站序列是AB，然后问你出站序列是不是可可？是否可以得到不同的出站序列？举个例子啊，那比如说这首先这里面是空的吧？我首先a进来对不对？好a进来那么呢？这个时候a进来里面已经oa了，这个时候a上面没有压住它的，所以这个时候a已经可以出站了。所以呢，这个时候就是边进可以边出，如果说a进来了，马上就出，那么这个时候出站序列就是a好，然后a出去了之后，这里面就没有了吧，然后b进来再b出去好，所以这样出站序列就得到AB。好，那么还有一种方式a进来，然后b进来，这个时候b就在最上面，对吧，然后b出去a出去，这样就得到的出站序列就是。BA所以你看啊，我们同样一个入站序列，可以得到得到两种不同的这个出站序列，当然你的元素更多，它就会得到更多种。不同的这个出站序列好

【核心概念：栈的定义】

- **本质**：**受限的线性表**
- 限制规则：**仅允许在一端（栈顶）进行插入与删除**，其余位置禁止操作
- 操作术语：**入栈**（插入）、**出栈**（删除）
- 结构端点：**栈顶**（可操作端）、**栈底**（封闭端）

【形象类比】

- 模型：**有底桶**
  - 桶底封死 → 元素无法从底部进入
  - 元素只能从桶口（栈顶）压入或弹出
  - 底部元素被上层元素压住，无法直接取出

【核心特性】

- 特点总结：**先进后出（FILO）** 或 **后进先出（LIFO）**，二者同义
  - 先进后出：最早进入者位于栈底，必须等上方元素全部出栈后方可离开
  - 后进先出：最后进入者位于栈顶，出栈时优先离开
- 生活实例：**一摞盘子/一摞书**（只能取最上方一个）

【考点：出栈序列的多样性】

- 关键结论：**相同入栈序列可得到不同出栈序列**
- 原因：**入栈与出栈操作可交替进行**（边入边出）
- 示例（入栈序列为 A、B）：
  - 方式一：A入 → A出 → B入 → B出 → 出栈序列 **AB**
  - 方式二：A入 → B入 → B出 → A出 → 出栈序列 **BA**
- 推论：元素越多，可得到的出栈序列种类越多

1栈 01:16:01

栈的概念与特点

【核心概念】

- **栈**：**受限制的线性表**
- 操作限制：**仅在一端（栈顶）进行插入/删除**
- **栈顶**：允许操作端
- **栈底**：另一端，不允许操作

【核心特点】

- **后进先出（LIFO）**，与**先进后出（FILO）**同义

出栈序列可能性分析（以d开头）

【题目条件】

- 入栈序列：a, b, c, d, e
- 要求：出栈序列以 **d** 开头

【推理过程】

- 以d开头 ⇒ **a、b、c、d 均已入栈**，d位于栈顶
- d出栈后，e尚未入栈，后续操作分情况讨论

【四种可能出栈序列】 - d, e, c, b, a

- e入栈出栈 → c → b → a
- **d, c, e, b, a**
  - c出栈 → e入栈出栈 → b → a
- **d, c, b, e, a**
  - c、b出栈 → e入栈出栈 → a
- **d, c, b, a, e**
  - c、b、a出栈 → e入栈出栈

【结论】

- 共 **4种** 可能序列，答案选 **B**

不可能的出栈序列判断

【题目条件】

- 入栈序列：a, b, c, d, e, f
- 要求：找出**不可能**的出栈序列（三可能一不可能）

【选项A分析】（不可能）

- 序列：f, e, g, d, a
- f出栈 ⇒ **a~f全部入栈**
- f、e依次出栈可行
- g入栈出栈可行
- d出栈可行
- **a出栈不可行**：a上方压有b、c，无法直接出栈
- **结论：A不可能，为答案**

【选项B分析】（可能）

- 序列：c, d, b, e, f, a, g
- c出栈 ⇒ a、b、c入栈
- d入栈出栈 → b出栈 → e入栈出栈 → f入栈出栈 → a出栈 → g入栈出栈
- **结论：B可行**

【选项C分析】（可能）

- 序列：e, f, d, g, c, b, a
- e出栈 ⇒ a~e入栈
- f入栈出栈 → d出栈 → g入栈出栈 → c、b、a依次出栈
- **结论：C可行**

【选项D分析】（可能）

- 序列：d, e, c, f, b, g, a
- d出栈 ⇒ a~d入栈
- e入栈出栈 → c出栈 → f入栈出栈 → b出栈 → g入栈出栈 → a出栈
- **结论：D可行**

【最终答案】

- **选A**

1) 例题:栈的定义 01:20:50

站是受限制的线性表

【核心概念】

- **栈**：**受限制的线性表**（逻辑结构）
- 本质：**线性表**，未确定存储结构

【顺序栈】

- 定义：**顺序存储**实现栈（工具：**数组**）
- 示意图：数组下标 **0~5**
- 关键变量：**top**（记录栈顶位置）

【入栈过程】

- 初始：栈空，**top指向0**
- 入栈a：a存入，**top上移一位**
- 入栈b、c：依次存入，**top继续上移**

【核心规定】

- **top指向栈顶元素的下一个空位置**（非栈顶元素本身）
- 优势：**top值 = 栈内元素个数**（如top指向3，则栈内有3个元素）

【出栈过程】

- 出栈c：c移除，**top下移一位**

【考点】

- **top指向规则**（下一个空位置）
- **top值与元素个数的对应关系**

2) 例题:栈的存储操作及指针变化 01:22:42

他说呢，这个顺序站，然后。然后呢呃呃零就是地址零作为占底，然后呢拓扑指向占顶，然后他说现在进站的操作的时候，这个拓扑指针怎么变化啊？就比如说。我们进一个c，然后这个就往上移动一个，对吧？就到这里，所以你会发现什么top就是由二变成三，所以是top+1个，对吧？加一个那就是加加嘛，

【核心概念】

- **顺序栈结构**：**地址0为栈底**，**top指针指向栈顶**
- **进栈操作**：**top指针上移**

【操作演示】

- 进栈元素：**c**
- 指针变化：**top由2变为3**
- 规律：**top+1**（即**top++**）

3) 例题:栈的存储操作及指针变化 01:23:04

栈的数组存储与入栈操作

【考点：栈的数组存储与入栈操作】

- **存储结构**：栈置于数组，容量为 n
  - 数组下标范围：**0 至 n-1**
  - 栈顶指针初始指向：**n-1**（题目明确指定；未指定时通常指向 0）
- **入栈操作分析**：
  - 空栈入栈 x：x 存放于下标 **n-1** 处
  - 入栈后 top 移动方向：**向下移动一位**（由 n-1 变为 n-2，**值减小**）
- **选项判断**：
  - 入栈语句特征：**top 值减小**（仅 C、D 符合）
  - 正确顺序：**先存 x 至 top 所指位置，再修改 top**
    - 原因：入栈前 top = n-1，x 应存于下标 n-1
    - 若先修改 top（top 变为 n-2），x 将错误存放于 n-2，**不符合入栈语义**

对顶栈（新考点）

【考点：对顶栈】

- 定位：**教材改版后新增内容**
- 要求：**简单了解**即可

- 核心概念：**对顶栈**（两栈共享同一存储空间，栈顶相对）

2.静态链表 01:24:46

堆顶站

【核心考点：两栈共享数组（双端栈）】

- **结构**：两个栈共用一个数组，从两端向中间入栈
- **栈顶指针**：左栈用 top左，右栈用 top右
- **入栈操作**：
  - 左栈入栈：元素从左侧放入，top左 向右移动
  - 右栈入栈：元素从右侧放入，top右 向左移动
- **互不影响**：两端独立操作，直至相遇
- **栈满条件**：top左 = top右 + 1（两指针相邻或交叉）

第三章的第四个考点链式栈

【核心考点：链式栈】

- **定义**：栈采用链式存储结构，即用单链表保存栈元素
- **带头结点与否**：均可，示意图以带头结点为例
- **栈顶位置**：由头指针指向的一端（即链表头部）
- **入栈操作**：只能在栈顶（头部）插入新节点
- **出栈操作**：只能删除栈顶（头部）节点

【考点：带头结点链栈的入栈操作】

- 已知：头指针 head，新节点指针 p
- 入栈本质：**单链表头插法**
- 两步操作：
  - **连接后继**：p->next = head->next（将p与原来第一个节点相连）
  - **更新头指针**：head->next = p（头结点指向新节点）

第三章的第五个考点队列

【核心考点：队列】

- **对比学习**：队列与栈可综合比较
- 队列特征：**先进先出（FIFO）**，栈为**后进先出（LIFO）**

三、队列 **01:29:09**

队列的定义与特征

【队列定义】

- **受限线性表**：与栈同为受限线性表，但限制条件不同
- **核心限制**：**一端插入，另一端删除**

【基本概念】

- **队尾**：允许插入的一端
- **队头**：允许删除的一端
- **入队**：在队尾插入元素
- **出队**：在队头删除元素

【形象理解】

- 类比：**两端开口的通道**（区别于栈的一端封死）
- 一端只能进，一端只能出

【操作示例】

- 入队顺序：a → b → c（均从队尾进入，依次排列）
- 出队顺序：a → b → c（均从队头离开）
- 逻辑类比：**日常排队**（先到先服务）

【核心特征】
- **先进先出（FIFO）**
- 等价表述：**后进后出**
- 考点：队列特征即**先进先出**

1.例题:队列的特征 **01:31:27**
队列的特点与相关判断题解析
【队列特点】
- **先进先出（FIFO）**：核心特征
- 受限线性表：**队尾插入、队头删除**
- 与栈对比：**栈为先进后出（FILO）**

【题目解析：队列特点选择题】
- 选项D：**先进先出** → 正确
- 选项A：**先进后出** → 栈的特点，错误
- 选项B：**只能插入** → 错误，**可插入也可删除**，仅限制插入位置在队尾
- 选项C：**只能删除** → 错误，**可删除也可插入**，仅限制删除位置在队头

【题目解析：队列概念判断题】
- 选项A：**队列属于非线性表** → 错误，队列为**受限制的线性表**
- 选项B：**队尾删除数据** → 错误，应为**队尾插入**
- 选项C：**队头插入** → 错误，应为**队头删除**
- 选项D：**先进先出** → 正确，即队列特点

四、顺序队列 **01:32:29**
第三章的第六个考点顺序队列
【顺序队列定义】
- **存储结构**：队列采用**顺序存储**（数组）→ 得**顺序队列**
- **与顺序栈区别**：队列**两端操作**，需**两个指针变量**；栈仅一端操作，只需栈顶指针

【指针变量定义】
- **front**：指向**队头元素**
- **rear**：指向**队尾元素的下一个空位置**（非队尾元素本身）

【指针移动规则】
- **入队**：**rear加1**（指向下一空位）
- **出队**：**front加1**（指向新队头元素）
- 初始状态：front = rear = 0（数组下标起始位置）

【模拟演示】
- 空队列：front、rear均指向0
- 入队a：front指向a（0），rear指向a下一空位（1）
- 再入队b、c：rear依次后移（2、3）
- 出队a：front后移至b（1），rear不变

【考点：出队操作语句】
- 核心操作：**front加1**
- 题目选项：出队语句为 **front = front + 1** → 选 **C**

五、循环队列 **01:35:03**

1.循环队列的概念与基本操作 01:35:05
第三章的第七个考点循环队列
【问题引入：假溢出】
- **场景**：数组容量已满（队尾指针指向末尾），但队首前仍有空闲空间

- **矛盾**：队尾指针无法再+1，直接入队会**溢出**
- **本质**：**假溢出**——空间未满，但指针无法回退
- **解决方案**：**循环队列**——将队尾指针回绕至数组头部，形成逻辑闭环

【循环队列核心机制】

- **指针移动**：入队时队尾指针+1，出队时队头指针+1
- **关键操作**：指针到达数组末尾时，**回绕到数组起始位置**（取模运算）
- **物理结构**：线性数组，逻辑上视为**环形**
- **核心优势**：**解决假溢出**，充分利用数组空间

【循环队列状态示意】

- **初始状态**：队头、队尾均指向起始位置
- **入队操作**：队尾指针后移，指向**最后一个元素的下一个空位**
- **出队操作**：队头指针后移，指向**下一个待出队元素**
- **循环回绕**：指针越过数组末尾时，**跳转至数组头部**继续移动

【核心考点】

- **循环队列本质**：仍是**队列**，遵循**先进先出（FIFO）** 原则
- **指针回绕**：通过**取模运算**实现（(rear+1) % MaxSize)
- **假溢出判定**：队尾到达物理末尾但队首前有空位，**并非真溢出**

2.循环队列的指针操作与条件判断 01:38:32

循环队列的入队、出队、空满判断与元素个数

【核心考点：循环队列指针移动规则】

- **入队操作**：rear = (rear + 1) % m（m为容量）
  - **统一分界处与普通位置**：如容量6时，5+1对6取余得0；2+1对6取余得3
  - 作用：**解决队尾指针在分界点（末尾→开头）的跳转问题**
- **出队操作**：front = (front + 1) % m
  - 与入队同理，**统一分界处头指针的移动**

【核心考点：队空与队满条件】

- **队空条件**：front == rear
  - 初始状态：两者同时指向0
  - 注意：**不一定非在0位置**，同时指向任意位置均表示空
- **队满条件**：(rear + 1) % m == front
  - 定义：**少存一个元素即为满**（用于区分队空与队满）
  - 示例：容量6，存0~4时，rear指向5，front指向0，此时(5+1)%6==0成立，即为满
  - 取余目的：**解决分界处（如5+1=6≠0）的判定问题**

【核心考点：循环队列元素个数计算】

- **一般情况**：元素个数 = rear - front
  - 示例：front指向3，rear指向5，个数为2
- **分界情况**：元素个数 = (rear - front + m) % m
  - 示例：front指向3，rear指向1（存3、4、5、0），(1-3+6)%6=4，个数为4
  - 公式统一：**(rear - front + m) % m 适用于所有情况**

3.例题1:循环队列的指针变化问题 01:42:02

循环队列的入队、判空、判满及指针移动

【核心考点：循环队列公式合并】

- 两种情形合并为**统一公式**：(尾指针 + 1) % 容量

- 目的：解决**假溢出问题**（尾指针指向 n-1 时再加一越界）

【考点：循环队列入队操作】

- 入队规则：**尾指针加一后对容量取余**
- 陷阱识别：数组下标范围 0 ~ m，则**容量为 m+1**（非 m）
- 正确语句：(尾指针 + 1) % (m+1)

【考点：循环队列判空条件】

- 判空条件：**头指针 == 尾指针**（头尾重合）
- 易错点：**必须用双等号**（==），单等号为赋值

【考点：循环队列判满条件】

- 判满条件：(尾指针 + 1) % 容量 == 头指针
- 原理：**尾指针再进一位即与头指针重合**

【考点：指针移动方向与元素位置判定】

- 入队方向：**尾指针沿增大方向移动**（如 4→5→0）
- 头指针指向**队头元素**，尾指针指向**队尾元素的下一个位置**
- 元素分布判定：从**头指针位置**沿增大方向至**尾指针前一位置**

【典型例题解析：容量为6的循环队列】

- 初始状态：容量 6，下标 0~5，尾指针 0，头指针 3
- 实际元素位置：**下标 3、4、5**（从头指针沿增大方向至尾指针前一位置）
- 操作过程：
  - **删除一个元素**：头指针后移一位（3→4）
  - **入队两个元素**：尾指针依次加一（0→1→2）
- 最终状态：**新尾指针指向 2，新头指针指向 4**

4.例题2:循环队列的元素个数计算 01:44:53

这个选b再看这个题啊，这个就是直接我们刚才分析过这个结论，就是怎样表示队列中的循环队列中元素的个数啊？这个你看这是零到m- 1，所以它容量是m好，我们说呃那个个数呢，就是尾减去头，然后要加上容量，再对容量取余，所以这个选a。

【考点：循环队列元素个数计算】

- **题干要点**：队列下标范围 **0 ~ m-1** → 容量为 **m**
- **核心公式**：元素个数 = **(尾 - 头 + 容量) % 容量**
- **本题结论**：直接套用公式 → **选 A**

5.链式队列的引入 01:45:13

第三章的第八个考点链式队列，那么我们前面讲过栈呃，如果说是保存呃，用这个保存在单链表里面，也就是链式存储，那就是链式栈啊，那么队列呢？

六、链式队列 **01:45:25**

1.链式队列的基本概念

链式队列的基本概念与操作

【链式队列定义】

- **本质**：单链表（链式存储）
- **核心指针**：
  - **front**：指向队头
  - **rear**：指向队尾
- **操作限制**：

- **插入**：仅允许在队尾（rear）操作
- **删除**：仅允许删除队头（front）首节点

【考点：带头节点的链队】

- 题目背景：**带头节点**的链式队列
- 关键点：**头节点**存在，front指向头节点还是首元节点需注意区分

2.例题1:链队列为空的条件 01:46:01

链表为空的条件

【核心考点：空链表判定条件】

- **判定方法**：**头指针与尾指针均指向头节点**
- **逻辑推导**：
  - 假设链表非空：存在三个元素
  - 链表为空：**所有元素节点均不存在**
  - 此时**尾指针回指头节点**
  - 结论：**头指针 == 尾指针**（均指向头节点）

3.例题2:链队列的入队操作序列 01:46:23

链队列入队操作（指针实现）

【题型定位】

- 场景：**链式队列**，指针 ss 指向新结点（元素为 x），执行**入队**操作
- 核心原则：**入队仅允许在队尾进行**

【操作步骤拆解】

- **第一步：链接新结点**
- 操作对象：**原队尾结点**
- 动作：将原队尾的 next 指针指向 ss
- 目的：**建立新结点与队尾的链接关系**
- **第二步：更新队尾指针**
- 操作对象：**队尾指针**
- 动作：将队尾指针 rear 移动至 ss
- 目的：**使新结点成为新的队尾**

【选项匹配】

- 正确操作序列：**先执行“尾结点 next 指向 s”，再执行“队尾指针指向 s”**
- 易错点：**两步顺序不可颠倒**，否则会丢失原队尾结点地址

4.例题3:不带头结点的单链表存储队列的删除操作 01:47:08

链队删除操作指针变化分析

【题型定位】

- 背景：**不带头结点的单链表**实现链队
- 操作：**删除（出队）操作**
- 核心问题：**队头、队尾指针是否修改**

【常规情况（队列元素≥2）】

- 示例：队列含 A、B、C 三元素
  - **头指针（front）** 指向 A
  - **尾指针（rear）** 指向 C
- 删除操作：**仅允许在队头进行**
  - 删除 A 后，front 移动指向 B
- 结论：
  - **队头指针必修改**

- **队尾指针不变**

【特殊情况（队列仅1个元素）】

- 场景：队列仅含 A
  - front 与 rear **均指向 A**
- 删除 A 后：队列变为**空**
  - front 与 rear **均置为 NULL**
- 结论：**队尾指针也可能需要修改**

【最终结论】

- 答案：**选 D**
- 判定依据：**队头指针必改，队尾指针可能改**（取决于队列是否只剩一个节点）

5.双端队列 **01:48:35**

第三章的第九个考点双端队列

【考点定位】

- **双端队列**：课程改版后**新增内容**
- 前置基础：**普通队列**（一端入队、一端出队）

【核心概念：双端队列】

- 定义：**两端均可进行插入和删除操作**的线性表
- 与普通队列对比：
  - 普通队列：**一端入队（队尾）、一端出队（队头）**
  - 双端队列：**两端皆可入队、两端皆可出队**

七、双端队列 **01:48:56**

双端队列及其受限形式

【双端队列定义】

- **两端均可入队、两端均可出队**
- 本质：突破普通队列单端进出限制

【输入受限的双端队列】

- 一端**禁止入队**，另一端可入队
- **两端均可出队**
- 核心特征：**入队受限，出队不受限**

【输出受限的双端队列】

- 一端**禁止出队**，另一端可出队
- **两端均可入队**
- 核心特征：**出队受限，入队不受限**

【考点提示】

- 区分两类受限队列的**限制端方向**（入队端 vs 出队端）
- 易混淆点：**输入受限≠输出受限**，需明确限制的是入队操作还是出队操作

八、双端队列应用 **01:50:15**

1.例题**1:**双端队列的输出序列判断

双端队列受限输入输出序列分析

【题型定位】

- 已知：**双端队列输入受限**（两种形式）
- 输入序列：**1、2、3、4**
- 目标：找出**两种受限队列均无法输出**的序列

【解题策略】

- 逐项验证：分别判断**左端输入受限**与**右端输入受限**两种模型

- 判定核心：**入队顺序固定，出队受出口位置约束**

【选项A：1 2 3 4】

- **左端受限**：按1→2→3→4输入，从出口依次输出，**可得1 2 3 4**
- 结论：**可达成**，排除

【选项B：2 3 1 4】

- **左端受限**：
  - 仅单出口，出队序列必为**4 3 2 1**
  - 问题转化：**两端可入，能否得2 3 1 4**
  - 分析：1入队后，2无论从哪端入，必邻接1，**无法插入3间隔**
  - 结论：**不可达成**
- **右端受限**：
  - 4出队 **⇒1、2、3、4全部入队**
  - 入队顺序固定：**1→2→3→4**
  - 出队过程：4出（右）→1出（左）→3出（右）→2出（左）
  - 结论：**可达成**
- 综合：**至少一种可达成**，排除

【选项C：1 3 2 4】

- **左端受限**：
  - 仅单出口，出队序列必为**4 3 2 1**
  - 问题转化：**两端可入，能否得1 3 2 4**
  - 分析：1入队后，2入队必邻接1，**无法隔3**
  - 结论：**不可达成**
- **右端受限**：
  - 4出队 **⇒1、2、3、4全部入队**
  - 入队顺序固定：**1→2→3→4**
  - 出队过程：4出（右）→后续需出2，但2**夹于1与3之间**，两端仅能出1或3，**2无法出队**
  - 结论：**不可达成**
- 综合：**两种均不可达成**，**符合题意**

【选项D：3 1 2 4】

- **左端受限**：
  - 仅单出口，出队序列必为**4 3 2 1**
  - 问题转化：**两端可入，能否得3 1 2 4**
  - 分析：1入→2从右入→3从左入→4从右入，**可达成**
- 结论：**可达成**，排除

【最终答案】

- **C**

2.栈和队列的应用 01:54:38

第三章的第十个考点栈和队列的应用啊，这个呢，你可以直接当结论记下来，那么栈。只要你看见关键字啊，递归和嵌套的时候，你就记住用栈就可以了啊，然后第二个队列用在什么时候，因为我们说队列我们不是讲过队列就跟你那个到排队打饭是一样的嘛，它很公平，这就是谁先来谁先排上，对吧？好，所以只要涉及到公平，也就是呃，保持这个良好的次序的时候呢，你就会用到这个队列，比如说我们进程调度啊，打印服务啊。哎，进程调度也是的，就是排队嘛，就是排队肯定是谁先谁先排上，最后就先调度谁嘛，打印也是的，你打印几个文件就是谁先等的，那最后就谁先打印嘛，好，我们看一下题目啊，这两个题目可以一起看，

【核心结论】

- **栈**：应用于**递归、嵌套**场景（直接记结论）
- **队列**：应用于**公平性、良好次序**场景（先来先服务）

【典型应用场景】

- **队列应用**：
  - **进程调度**（排队调度，先到先调度）
  - **打印服务**（先等待先打印）

九、栈和队列的应用 **01:55:22**

栈和队列的应用（习题讲解）

【考点：栈与队列的应用场景】

- **栈**：**递归、嵌套**（函数调用、括号匹配等）
- **队列**：**进程调度、打印任务**（保证公平次序）

【习题解析】

- **题干含“嵌套”“递归”**
- 定位：**栈的应用**
- 排除：进程调度（队列应用）
- 答案：**B**
- **题干问“嵌套和递归用什么处理”**
- 答案：**栈**
- **题干含“保证某种次序/公平”**
- 定位：**队列的应用**
- 排除：栈（嵌套、递归）
- 典型场景：**进程调度、打印**
- 答案：**D**
- **题干“进程调度”**
- 目的：**保证公平**
- 应用结构：**队列**
- 答案：**C**

第四章 数组

【章节导入】

- 研究对象：**数组**
- 基础：**一维数组**（相对熟悉）

十、数组 **01:56:27**

1.数组的基本概念与下标 01:56:28

一维数组与二维数组的基本概念

【一维数组】

- **定义**：容量为 n 的数组 a[n]
- **下标范围**：**0 到 n-1**（从 0 开始）
- **形象理解**：**一排连续内存格子**，每格存放一个元素
- **示例**：a[4] → 元素为 **a[0]、a[1]、a[2]、a[3]**

【二维数组】

- **构造逻辑**：一维数组的每个元素**替换为一个一维数组**
- **本质**：**数组的数组**（元素本身为一维数组）
- **结果形态**：由一维数组嵌套构成，形成**二维结构**

2.二维数组及其存储方式 01:57:14

二维数组的定义与存储方式

【二维数组定义】

- **本质**：一维数组的元素为一维数组时，构成二维数组
- **结构示例**：四行四列（a00~a33）
- **下标规则**：**仍从0开始**
- **扩展性**：元素可继续存放一维数组 → **三维、四维数组**（考试仅需掌握一维、二维）

【二维数组存储方式】

- **存储本质**：二维数组**仍存储于一维数组**中
- **两种存储方式**：
  - **行主序存储**：按行依次存放（第一行存完→第二行→第三行→第四行）
  - **列主序存储**：按列依次存放（第一列存完→第二列→第三列→第四列）

3.偏移量的概念与计算 01:58:43

接下来呢，介绍一个我们经常会提到的一个概念，叫做偏移量啊，什么叫偏移量呢？实际上，它就是表示间隔多少个，咱们举个例子啊，比如说。我现在问这个在这个里面a1和a1对a0的偏移量多少，实际上就是看a1和a0间隔多少个，这个时候它间隔一个吧，所以它的偏移量是一好，那么a3。对a0的间隔呃，偏移量是多少？你看a3到a0间隔几个，这是间隔一个吧，然后间隔两个。间隔三个，所以a3对a0的偏移量是三，那么呢，对于一个一维数组，我只需要知道这个第一个它的地址是多少，其他的每一个元素我知道。对第一，相对于第一个的偏移量是多少，然后乘以每一个格子的这个的存储单元，哎，那我就能够计，这样就可以计算出其他任意一个元素的这个地址，那么同理。这个在这个二维数组里面也可以同样应用啊，因为我们说二维数组把它存存起来的时候，它仍然是一个一维数组，也就是说呢，这边这个二维数组啊，它我们存的时候，你看这边投。这呃，这边第一行的尾是不是就这样又接了这边头了，这边这边又这样接过来了，对吧？这样又接过来了，所以它实际上仍然是一个一维数组好，那么假设说我要。计算这个问他的地址是多少，我只需要知道他和第一个第一个的偏移量是多少就可以了，那么呢，像这个二维数组，它这个偏移量我们怎么找？好有两种方法，第一种方法呢，就是它元素给的比较少的时候，就比如说我们现在这个，我们给的这个二维数组，它元素比较少，你可以把它整个画出来，你可以一个个数嘛。对吧，你可以一个个数看它，跟它相隔多少个，你看啊，我这个我就可以数一下，你看这个是相隔一个吧，这相隔两个相隔三个好相隔三个，然后下面每下面每一行都是相隔四个，那就是呃，下面就三四一十二加上。三所以所以这个最后一个和第一个就是相隔15个，所以偏移偏移量是15好，所以那就用第一个的地址，再加上15乘以每个格子的这个存储单元就可以了，那么第二种方法啊，如果说他这个。元素比较多，就说你很难把它整个画出来的时候，我们怎样找规律啊？我们就假设它是一个m行n列的，一个呃二维数组哈，然后呢，这里可以告诉一个什么规律呢？就是它的上下两行相同位置，也就是这个a0和a1哎，它是间隔n个也就是这个，它有多少列，它就是间隔它的偏移量就是n，所以呢，这个这个地方它有四列嘛，所以它的所以这个a0。零和a一零哎a一零对这个a零零它就是见呃偏移量为四，那么呢，这个a三零相对于这个a零零它一共间隔了三行吧，然后每一行都是四，那这就是三四一十二哈。然后你再看啊这个，然后你再看最后一行，他和他和他间隔多少个，这是间隔一个间隔两个间隔三个，所以就是前面的三四一十二，

【核心概念：偏移量】

- **定义**：表示元素间**间隔的个数**
- 示例：a1对a0偏移量为**1**；a3对a0偏移量为**3**

【一维数组地址计算】

- 原理：已知**首元素地址** + 各元素**相对偏移量** × **存储单元大小** = 任意元素地址

【二维数组地址计算】

- 存储本质：二维数组**按行存储**，物理上仍为**一维数组**
- 偏移量求解方法：
  - **方法一（元素少）**：**画图逐个数**，统计间隔个数
    - 示例：4列数组，末元素与首元素间隔**15个**，偏移量为**15**
  - **方法二（元素多）**：**找规律推导**
    - 规律：**上下行同列元素**（如a[i][0]与a[i+1][0]）偏移量为**列数n**
    - 示例：4列数组，a[1][0]对a[0][0]偏移量为**4**
    - 跨行计算：a[3][0]对a[0][0]偏移量 = **3行 × 4列 + 3** = **15**

4.例题1：二维数组的地址计算 **02:01:28**

二维数组地址计算（续）

【考点：二维数组元素定位（下标从0开始）】

- 示例：三行四列数组
  - 未显式赋值元素默认为 **0**
  - 行0：3，4，0，0
  - 行1：2，8，6，0
  - 行2：0，0，0，0
- 求 a[1][2]：
  - **下标从0开始**（行0、1、2；列0、1、2、3）
  - 定位：**第1行第2列** → 值为 **6**
  - 答案：**B**

【考点：行主序地址计算公式推导】

- 场景：十行五列数组（m=10，n=5），已知 a[0][0] 地址为1000
- 规律：
  - **相邻行同列元素间隔 = 列数n（即5）**
  - 行偏移：a[i][0] 与 a[0][0] 间隔 **i个n** → 偏移量 = **5i**
  - 列偏移：a[i][j] 与 a[i][0] 间隔 **j个元素** → 偏移量 = **j**
- 公式：**地址 = 1000 + 5i + j**
- 答案：**C**

【考点：大规模数组规律应用】

- 场景：十行二十列数组
- 处理策略：**元素过多无法画图，必须直接套用行主序地址公式**

5.例题3：二维数组的地址计算 **02:03:50**

好，然后每个元素占四个单元，然后呢，他告诉了这个手呃a零零的地址是2000，然后他问这个，那我首先要找到他吧，那么第二行第一个是a一零。然后a二零a三零a四零a五零好，那么就在这一行，然后往后a五一，然后往后。到a5。十好，所以实际上就是要问什么呢？问最后一个到它的这个偏移量，当然这个题里面它没有说是按照行主序啊，但它一般没题的时候我们就默认它是行主序好，实际上现在问它的偏移量是多少？然后它的这个它的地址是2000，那么根据我们的规律啊，就是同一行上下间隔的，因为它是十行20列嘛。好同一行间，这个偏移量是列数，所以这里就间隔20，那你就看这有几个20，这是一个两个三个。四个五个，所以这个到所以它到它是间隔五个20啊，所以它的偏移量是五×20。然后再加上看这里到这里的偏移量是多少，那就用这个减去它，所以就是十个偏移量，那就是十好，这是总的偏移量，然后每个元素。占四个，所以就乘以四嘛，然后再加上手地址2000好，那这个式子列出来了，那就变成2000，加上这里面是。110那就是440吧，440，所以这就得到二四四零。第四章的第二个考点，

【考点：二维数组地址计算（行主序）】

- **题目条件**
- 数组规模：**10行×20列**
- 首地址 A[0][0] = **2000**
- 每个元素占 **4** 个存储单元
- 默认存储方式：**行主序**
- **目标元素定位**
- 目标：A[5][10]
- 行定位：第5行（A[5][0] 起始）
- 列定位：第10列
- **偏移量计算**
- 行间偏移：从 A[0][0] 到 A[5][0]，间隔 **5行**
  - 每行偏移量 = 列数 = **20**
  - 行偏移总量 = **5 × 20 = 100**
- 列间偏移：从 A[5][0] 到 A[5][10]，间隔 **10列**
  - 列偏移量 = **10**
- 总偏移量（元素个数）= **100 + 10 = 110**
- **地址求解**
- 地址公式：**首地址 + 总偏移量 × 每个元素占单元数**
- 计算：2000 + 110 × 4 = 2000 + 440 = **2440**
- **结论**
- A[5][10] 的存储地址 = **2440**

6.对称矩阵的概念与特点 02:05:14

咱们介绍一种特殊的矩阵，叫做对称矩阵啊

【对称矩阵定义】

- **前提条件**：必须是**方阵**（行列数相等）
  - 反例：三行四列、四行三列均非方阵

【主对角线概念】

- **主对角线**：左上至右下连线
- **负对角线**：右上至左下连线
- 易混淆点：对称轴为**主对角线**，非负对角线

【对称矩阵判定】

- 判定标准：方阵元素**沿主对角线对称**
- 示例：三阶方阵中，a_{12}与a_{21}、a_{13}与a_{31}、a_{23}与a_{32}对应相等
- 反例：若对应位置元素不等，则非对称矩阵
- 修正：将对应位置改为相同数值（如2、3、6），即构成对称矩阵

7.对称矩阵的符号表达 02:06:30

对称矩阵的符号表达与存储优化

【对称矩阵的符号表达】

- 矩阵下标：**从0开始**（第0行、第1行、第2行）
- 对称矩阵定义：**元素下标互换后值相等**
  - 例：m[1][2] == m[2][1]
  - 本质：**下标互换，元素值不变**
- 对角线元素：i == j，互换后仍为自身，**必然相等**

- 例：m[0][0]、m[1][1]、m[2][2]

【核心考点：对称矩阵的存储优化】

- 矩阵本质：**二维数组**
- 对称矩阵特点：**约一半元素与另一半对称相等**
- 存储策略：**仅存储上三角（或下三角）部分**
  - 未存储部分：通过**对称性映射**直接获取对应值
  - 效果：**节省近一半存储空间**

8.例题4：对称矩阵的表示 **02:07:53**

第四章第三个考点：特殊矩阵（对称矩阵）

【考点：对称矩阵】

- **定义**：**行列下标互换后元素仍相等**的矩阵
- **数学表达**：$a_{ij} = a_{ji}$（对所有 i, j）
- **核心性质**：**矩阵与其转置相等**，即 $A = A^T$
- **存储优化**：**仅需存储上三角或下三角部分**，其余元素可省略，**节省存储空间**
- **考查方式**：直接考查对称矩阵的**等式判定**（互换下标后相等）

9.三角矩阵的概念与特点 02:08:11

三角矩阵

【定义与分类】

- **三角矩阵**：**方阵**（仅方阵有对角线）
- 细分：**上三角矩阵**、**下三角矩阵**

【上三角矩阵】

- 定义：**主对角线以下元素全为零**
- 特征：非零元素集中于**主对角线及其上方**，呈三角形

【下三角矩阵】

- 定义：**主对角线以上元素全为零**
- 特征：非零元素集中于**主对角线及其下方**，呈三角形

【核心特点：压缩存储】

- 矩阵本质：**二维数组**
- 存储优化：**零元素无需存储**，仅保存非零元素
- 目的：**节省存储空间**

10.三角矩阵的元素个数计算 02:09:14

三角矩阵元素个数计算

【核心考点：三角矩阵元素个数公式】

- **结论**：n行n列三角矩阵（上/下三角）元素总数 = **n(n+1)/2**
- **推导方法一（等差数列求和）**：
  - 各行元素数：**1, 2, 3, ..., n**
  - 求和公式：**（首项+尾项）×项数÷2** = (1+n)×n÷2 = **n(n+1)/2**
- **推导方法二（整体减半法）**：
  - 总元素数：n^2
  - 对角线元素数：**n**（每行1个）
  - 非对角线部分：$n^2 - n$，上下三角**平分**
  - 下三角元素数 = $(n^2 - n)/2 + n = n(n+1)/2$
- **易错点**：两种方法结果一致，需注意**对角线元素归属**及**平分逻辑**。

11.例题5：三角矩阵的压缩存储 02:10:53

加n跟刚才这个算的是一样的好，我们来看一下题目啊，这个就是呃三角矩阵的概念，他说以什么为界，那就是主对角线为界呃，有上面或者下面一半呢。是零，当然这里他还说可以，其实不光可以是零，还可以是其他数字，比如说全部是一，全部是二也可以啊，实际上和零是一个呃，是是类似的啊，这样就叫做三角矩阵，再来看这道题啊。它这个说了是一个下三角矩阵，然后这是十行十列，然后把它压缩存储到一个呃一维数组里面，它没有说是行主序还是列主序的时候，我们就当它是行主序啊就是。呃呃，一行一行的存，然后他问这个里面这个元素，它的下标是多少？实际上呢，他就是问这个在这个一维数组里面排排第几个啊？好，所以我们就把这个画一下啊。好，那么它是一个下三角，所以呢，它是a第一行就一个嘛a零零，然后AA一零，第二行两个a一零a一一。然后a二零a二一a二二好，就按照这个规律啊呃，当然这个因为它给的不多，其实你可以把它全部全部写出来啊a3。零然后a四零那么这个就在a四零后面啊，那就是a四一a四二。a四三好，所以呢，我们的目标在这里，那么现在呢？我们这个把它存在这个数组里面，那么第一行就是一个。第二行两个，第三行三个，第四行四个好，所以这前面这一共存了几个，这个就是一+2+3+4，这是十个，这是十个好，那么这是第11个，12个，13个，14个，所以它是14个，那么呢，第十。14个，因为我们这个一维数组是从a0开始的，对吧？好，所以这个下标为零是呃，第一个那么它是第14个，所以它的下标是13好，所以。它问的是下标位置嘛，所以这个是13好选c。

【三角矩阵概念】

- **定义**：以**主对角线为界**，上三角或下三角部分为**零**
- **扩展**：非零元素可为**任意常数**（如全1、全2），与零本质**类似**

【下三角矩阵压缩存储】

- 题干：**10行10列**下三角矩阵，压缩至**一维数组**
- 默认存储方式：**行主序**（一行行连续存储）
- 目标：求元素 **a_{43}** 在一维数组中的**下标**

【存储过程推导】

- 下三角逐行元素分布：
  - 第1行：a_{00}（1个）
  - 第2行：a_{10}、a_{11}（2个）
  - 第3行：a_{20}、a_{21}、a_{22}（3个）
  - 第4行：a_{30}、a_{31}、a_{32}、a_{33}（4个）
  - 第5行：a_{40}、a_{41}、a_{42}、**a_{43}**（目标）
- **前4行元素总数**：1+2+3+4 = **10个**
- a_{43} 位置：第**11、12、13、14**个元素
- **一维数组下标**：从0开始，第14个元素下标为 **13**

【结论】

- 答案：**C（下标13）**

12.稀疏矩阵的概念与特点 02:12:43

第四章的第四个考点，再介绍一种特殊的矩阵，叫做系数矩阵啊呃，这个系数矩阵我们前面讲的这个对称矩阵和这个三角矩阵，它必须是方阵系数矩阵，不需要是方阵啊。因为我什么时候会用到方阵呢？就是你要用到对角线的时候才才需要是方阵这个系数矩阵不涉及到对角线，它不需要是方阵啊，当然我们这边给了一个例子。它是一个方阵，但是不不代表说它必须是方阵嗯，稀疏矩阵，简单来说就是呃，非零元素非常少的矩阵，比如说我们这一个三×3的矩阵里面非零元素只有两个。只有两个啊，那么其他大部分是零啊，所以它的非零元素比较少，所以它就是一个稀疏矩阵，那么这

个稀疏矩阵有什么特点呢？呃，特点就是呃，非零大部分是零零非常少啊，那我们自然。在它保存它的时候呢，我们就没有必要把那些零都保存了啊呃，和那个三角矩阵一样，那些零我们都不用保存了，我们就把这些个非零元素给它挑出来保存。

【核心考点：稀疏矩阵】

- **定义**：**非零元素极少**的矩阵
- **与对称/三角矩阵区别**：
  - 对称/三角矩阵：**必须是方阵**
  - 稀疏矩阵：**不要求方阵**（不涉及对角线）
- **判定标准**：非零元素占比极低（如3×3矩阵仅2个非零元素）
- **存储策略**：
  - **仅保存非零元素**，零元素全部舍弃
  - 与三角矩阵存储思路一致

13.稀疏矩阵的三元组表表示 02:13:42

三元组表的概念与结构

【核心概念】

- **三元组表**：**稀疏矩阵压缩存储**的核心结构
- 定义：仅提取**非零元素**，记录其**行、列、值**三项信息

【三元组构成】

- **i**：非零元素所在**行号**
- **j**：非零元素所在**列号**
- **v**：非零元素的**数值**

【存储示例】

- 矩阵规模：**3行3列**
- 非零元素：**2个**
- 提取过程：
  - 元素“1”：**第0行，第0列**，值=1
  - 元素“3”：**第2行，第2列**，值=3
- 保存内容：**矩阵总行数、总列数、非零元素个数**（即：3、3、2）

【关键点】

- 三元组表**仅存非零元素**，跳过零元素，实现**空间压缩**

14.例题6：稀疏矩阵的表示 02:14:44

这个是另外保存的好，我们来看一下题目啊，这两个概念题一起来看一下是呃，你看这啊，它说m行n列，你看我们刚才说的它不需要是这个方阵吧m1，如果说是方阵。它就n行n列啊，它是有t个非零元素，现在这个t两个小于号是表示远远小于的意思啊，远远小于它总的这个元素个数的时候，我们就把这个叫做系数矩阵。然后下面一个题稀疏矩阵可以用什么来这个进行这个存储，那么这里就填三元组表，再看这个题啊，三元组表的目的是什么？其实和那个上三呃三角矩阵是一样的啊，就是那些零。呃，

【概念辨析：稀疏矩阵】

- **矩阵维度**：**m行n列**，不要求为方阵（方阵特例：n行n列）
- **非零元素个数**：**t个**
- **稀疏判定**：**t 远小于 总元素个数**（符号：**<<**）
- **定义**：满足上述条件者，称为**稀疏矩阵**

【考点：稀疏矩阵存储方式】

- 存储方法：**三元组表**

【考点：三元组表目的】

- 核心目的：**压缩存储零元素**（与三角矩阵压缩存储目的相同）

15.例题7：稀疏矩阵的表示 **02:15:18**
都是重复的，没必要存啊，所以呢，就是节省空间，所以这个选a再看一下这道题啊，哎，我们这个就比我们给的那个例子要元素多一点，但是仍然是一样的啊，就是把它的三元组表。写出来好那么三个值I行值列值和它的数值好，然后呢？然后我们来来里面找啊，那你看。呃，我们先把它这个行列给它标出来，那么是零一二三四，然后零一二三四五。好，然后我们来看啊，那么这个二它是零零行一列吧？就零一二好，然后接着。接着这个五它是二二行二行三列五，然后这个九的话是二行五列二行五列九。那么，这个六是三行一列，三行一列。六好，然后这个四是四行四列，四行四列，这个四行四列四好，所以这很简单。第四章的第五个考点，
第四章第五个考点：稀疏矩阵三元组表构建
【核心考点】
- **三元组表定义**：仅存储**非零元素**，节省存储空间
- 三元组结构：**（行下标，列下标，数值）**

【题目解析】
- 矩阵维度：**4行5列**（行下标0-3，列下标0-4）
- 非零元素定位：
  - **2** → **(0, 1)**
  - **5** → **(2, 3)**
  - **9** → **(2, 5)**
  - **6** → **(3, 1)**
  - **4** → **(4, 4)**
- 构建三元组表：按**行优先顺序**依次列出各非零元素的三元组

【方法总结】
- 步骤：**标注行列下标 → 逐个提取非零元素 → 填入三元组表**
- 关键点：**行、列下标从0开始计数**

16.广义表的概念与特点 **02:16:19**
广义表的引入与定义
【广义表概念引入】
- **来源**：线性表概念的**扩展化**
- **线性表定义**：元素**一个接一个**排列，且**必须同类型**
- **广义表核心**：**放宽同类型限制**，允许元素**本身也是线性表**

【广义表结构示例】
- 原线性表：(1, 2, 3)
- 变化：将元素 2 替换为线性表 (1, 2, 3)
- 新结构：(1, (1, 2, 3), 3)
  - 第**1**个元素：1
  - 第**2**个元素：**子线性表** (1, 2, 3)
  - 第**3**个元素：3

【核心定义】
- **广义表**：线性表中的元素**仍可为线性表**，即**表里套表**
- 本质：**递归定义**的线性表结构

17.递归表的概念 **02:17:29**
递归表的定义与特征
【核心考点：递归表】

- **定义**：**自身作为自身元素**的广义表（自己套自己）
- **示例**：广义表 g = (1, 2, g)
  - 元素构成：**第一个元素为1**，**第二个元素为2**，**第三个元素为g（即表自身）**
- **本质特征**：**递归嵌套**，表内某元素直接或间接引用该表本身

18.广义表的长度 02:17:50
广义表的长度
【核心考点：广义表长度】
- **定义**：广义表**最外层括号内直接元素的个数**
- **求法**：去掉最外层括号，按**逗号分隔**计数
- **示例**：
  - 广义表 (a, b, c)：长度 = **3**
  - 广义表 (a, (b, c), d)：长度 = **3**（第二项整体算一个元素）
- **易错点**：元素本身可多层嵌套，但计数只看**最外层逗号分隔的段数**
- **特殊情况**：**空表长度 = 0**（无任何元素）

19.例题8：广义表的长度 02:18:38
数组长度与元素个数
【核心考点】
- **数组长度 = 元素个数**
- 判定依据：**逗号分隔**的独立数据项
【实例解析】
- 示例一：逗号分隔出 **4个元素** → 长度 = **4**
- 示例二：表长为 **3** → 内含 **3个元素**

20.例题9：广义表的长度 02:19:00
我们来看一下，你看第一个这里面。整个是一个元素，对吧？所以它的长度为一好，再看这个b，你看这里分开，然后这里分开，然后这里分开，所以它是三个元素，它是为三，所以这个选这个啊。再看CC的话，这里分开，这里分开，这里分开，然后最后这一个，所以这里长度为四，然后这最后一个，这里分开，这前面一个，这后面一个，所以这长度为二，
【数组长度判定】
- **核心方法**：依据**逗号分隔**统计元素个数
- **示例解析**：
  - **a**：整体为单一元素 → **长度 = 1**
  - **b**：三处逗号分隔 → **长度 = 3**
  - **c**：三处逗号分隔，末段含独立元素 → **长度 = 4**
  - **末项**：两处逗号分隔 → **长度 = 2**

21.例题10：广义表的长度 02:19:21
所以这个选b，再看这道题啊。看这是不是一个空表，这不是一个空表啊，空表是一层括号啊，那么这个把外面括号去掉之后，这里面里面是一个空表，但是这整个里面就看成一个元素，对吧？所以里面有一个元素，所以是长度为一。
【题目解析】
- 题干：判断是否为**空表**，并求**表长**
- 关键概念：
  - **空表**：仅为一层括号，即 ()，内部无任何元素
  - **非空表**：括号内存在元素（元素可为原子或子表）

- 本题结构：
  - 外层括号去除后，内部为**一个空表** ()
  - 该空表整体视为**一个元素**
- 结论：
  - 该表**不是空表**
  - **表长 = 1**（仅含一个元素）

22.广义表的深度 02:19:38

第四章的第六个考点

【考点：广义表深度】

- **定义**：广义表**括号嵌套的最大层数**
- **空表深度**：**1**（仅最外层括号）

【深度计算要点】

- **易错点**：**勿漏最外层括号**（计为第1层）
- **递归表深度**：**无穷大**（自身嵌套无限循环）

【例题解析】

例1：广义表 (a, b, (c, (d)))

- 元素划分：3个元素（第3个为嵌套子表）
- 深度计算：
  - 第1层：最外层括号
  - 第2层：第3个元素的外层括号
  - 第3层：子表 (d) 的括号
- 结果：**深度为3**（选B）

例2：广义表 (a, (b, (c, (d))), e)

- 元素划分：3个元素
- 深度计算：
  - 第1层：最外层括号
  - 第2层：第2个元素的外层括号
  - 第3层：(c, (d)) 的括号
  - 第4层：(d) 的括号
- 结果：**深度为4**（选B）

例3：递归表

- 深度：**无穷大**
- 区分：**长度**（元素个数）与**深度**（括号层数）不同，递归表长度为2，深度为无穷

23.广义表的表头和表尾 02:21:43

第四章的第七个考点，咱们继续介绍广义表的两个概念，叫做表头和表尾，好表头是什么意思呢？就是表中第一个元素，比如说这边咱们有一个广义表，里面有三个元素啊，它的第二个元素是表，里面又套了一个。表好，那么这个时候它的表头什么就是指它的第一个元素，所以这个时候它的表头就是一，那么什么叫表尾呢？表尾就是把这个广义表。去掉第一个元素，我就把这个一去掉那么呃，一去掉那么剩下就是由两个元素组成一个广义表，这个就叫表尾注意表尾不是说最后这一个元素啊，那么呢，我们有的时候会用到这个取表头和取表尾的操作。就是这样表示啊，就是head，然后这个里面括号里面就是对广义表，比如说这个广义表是积啊，那么这个就表示对积取表头，那么这个取表头取了，就是刚才我们取表头就是一吧。然后这个对积取表头就

是取表尾，那么就得到什么得到是把一去掉剩下的一个广义表好，我们来看一下题目啊，

【核心概念】广义表：表头与表尾

- **表头**：广义表的**第一个元素**
- **表尾**：**去掉第一个元素后**，由剩余元素组成的**广义表**
  - 易混淆点：**表尾 ≠ 最后一个元素**

【取操作表示】

- **取表头**：Head(广义表) → 返回**第一个元素**
- **取表尾**：Tail(广义表) → 返回**去除首元素后的广义表**

【示例说明】

- 广义表 A = (1, (2, 3), 4)
  - Head(A) = **1**
  - Tail(A) = **((2, 3), 4)**（仍为广义表）

24.例题11：广义表的表头和表尾 02:22:39

广义表取表头取表尾操作示例

【核心考点】

- **取表头**：广义表的**第一个元素**（可为原子或子表）
- **取表尾**：**去掉表头元素后剩余部分**（注意：非取最后一个元素）

【示例一：连续操作】

- 原表：(a, b, c, d)（4个元素）
- 操作链：
  - **取表尾**：去掉表头a → 剩余(b, c, d)
  - **再取表尾**：去掉表头b → 剩余(c, d)
  - **取表头**：取第一个元素 → **c**

【示例二：多层嵌套连续操作】

- 原表：(a, b, (c, d), e)（4个元素）
- 操作链：
  - **取表尾**：去掉表头a → 剩余(b, (c, d), e)
  - **再取表尾**：去掉表头b → 剩余((c, d), e)
  - **取表头**：取第一个元素 → **(c, d)**（子表整体）
  - **再取表尾**：去掉表头c → 剩余(d)
  - **取表头**：取第一个元素 → **d**

【易错提醒】

- 取表尾结果**仍为广义表**（即使只剩一个元素）
- 取表头结果**可为原子或子表**，取决于原表结构
- 连续操作时，**每步操作对象为当前剩余广义表**，需重新识别表头/表尾

25.字符串的概念与特点 02:24:10

第四章考点八：字符串（串）的基本概念

【核心定义】

- **字符串（串）**：由**字符组成的有限序列**
- 字符类型：**字母、数字、其他符号**
- 示例："AB125"（5个字符组成的序列）
- 命名方式：可赋名，如 s = "AB125"

26.字符串的长度与空串 02:24:40

字符串长度与空串概念

【核心概念】

- **字符串长度**：指字符串中**字符的个数**
  - 示例：给定字符串含**5个字符**，则长度为**5**
- **空串**：**长度为零**的字符串
  - 特征：**内部不含任何字符**
  - 示例：s1 为空串，即 ""，**无任何内容**

27.字符串的子串 02:25:01

子串的概念

【核心考点：子串定义】

- **子串**：字符串中**连续**取出的**任意长度**字符序列
- **主串**：包含子串的原始字符串
- **关键约束**：**必须连续**，不可跳跃选取
  - 反例：取 A 与 2（中间跳过 B、1）→ **非子串**
  - 正例：AB、AB12 → 均为子串

【两个特例】

- **空串**：长度为0，**是任意字符串的子串**
- **自身**：完整取出整个字符串，**是自身的子串**

28.例题12：字符串的子串 02:26:03

子串数量计算

【题目信息】

- 给定主串：**8个字母**
- 求解目标：**所有不同子串的总数量**

【解题思路：按长度分类计数】

- **长度0（空串）**
- 数量：**1种**
- **长度1**
- 每个字母单独成串
- 数量：**8个**
- **长度2**
- 连续取2个字母，每次向后移动1位
- 移动次数：**7次** → 数量：**7个**
- **长度3**
- 移动次数：**6次** → 数量：**6个**
- **长度4**
- 数量：**5个**
- **长度5**
- 数量：**4个**
- **长度6**
- 数量：**3个**
- **长度7**
- 数量：**2个**
- **长度8**
- 取整个串：**1个**

【求和计算】

- 长度1至8：**等差数列**（1到8）
- 求和公式：**(首项+尾项)×项数÷2**
  - (1+8)×8÷2 = **36**
- 加上空串：36+1 = **37**
- 最终答案：**D（37）**

十一、树 **02:27:39**

第五章的第一个考点树

【树的定义与特点】

- **逻辑结构**：四种基本逻辑结构之一（第三种）
- **核心特征**：**层次化结构**，自上而下分层
- **层数编号**：从**第0层**开始（非第1层）

【根结点】

- 定义：**第0层结点**，树的起源
- 性质：**一棵树有且仅有一个根结点**
- 分支规则：根结点向下分支数**不限**（0个、1个、多个均可）

【基本术语】

- **父结点（父型）**：某结点向下分支出的结点，该结点为其子结点的父结点
  - 例：A是B、C、D的父结点；C是F、G的父结点
- **孩子结点**：某结点分支出的后继结点
  - 例：B、C、D是A的孩子；F、G是C的孩子
- **兄弟结点**：**同一父结点**分支出的结点互为兄弟
  - 例：B、C、D互为兄弟；F、G互为兄弟
  - 易错：**F、G与E不是兄弟**（父结点不同）
- **子树**：某结点及其所有后继结点构成的整体
  - 例：A分出3棵子树（B子树、C子树、D子树）

【空树】

- 定义：**一个结点都没有**（根结点也不存在）

【结点的度】

- 定义：**某结点向下分支的数量**（即孩子个数）
  - 例：A的度为3；C的度为2；D的度为0；B的度为1

【树的度】

- 定义：**所有结点度的最大值**
  - 易错：**不是所有结点度之和**
  - 例：该树中最大度为3（A），故树的度为3

【叶结点（叶子）】

- 定义：**度为0的结点**（向下无分支）
- 形象理解：位于树的最末端，如同真实树叶
  - 例：D、E、F、G均为叶结点

【树的深度】

- 定义：**根结点在第0层**，树的最大层数即为深度
  - 例：该树最大层数为第2层，深度为**2**

【树的高度】

- 定义：**深度 + 1**（即树的总层数）
  - 例：深度为2，高度为**3**

【森林】

- 定义：**两棵或两棵以上**的树组成森林
- 易错：**单棵树不构成森林**

1.例题:树的结构描述 02:32:48

这三个在一起也可以叫森林好，我们来看一下题目啊，首先第一呃，这个题它是关于树的描述，正确的是a选项，它说树是线性结构啊，我们四种逻辑结构，树是非线性结构吧，所以这个错了啊，然后b和d。可以对照的一起来看啊，他说树中每个节点可以有多个直接前驱啊，也就是也就是说呃，一个节点向上可以有多个我们是向上不能有多个的啊，也就是说向上只能有一个向下可以有多个。好呃，也就是一个节点的副节点只有一个啊，也是他父亲总只有一个，但是他的孩子可以有多个好，所以这里说有多个呃，上一层有多个，这是不对的啊，然后他说只能有一个直接后取向下他的意思就是向下只有一个也是不对的啊，就是。向下分可以分好几支啊，所以这三个都错了，那么c是对的，树是逻辑结构吧，树的存储结构还没定，那么树可不可以存在数组里面顺序存储是可以的啊，再看这道题叶子叶子就是我们说的叶结点嘛，叶结点就是向下没有分支，就是它没。没有孩子啊，然后度就是他有几个分支好，那么所以夜绩点度就为零，所以这个选b好，

【考点：树的描述判断题】

- **A选项**：树是线性结构 → **错误**
  - 依据：四种逻辑结构中，**树属于非线性结构**
- **B选项**：每个节点可有多个直接前驱 → **错误**
  - 规则：**向上（前驱/父节点）仅一个**
- **D选项**：每个节点仅一个直接后继 → **错误**
  - 规则：**向下（后继/孩子）可有多个**
- **C选项**：树是逻辑结构 → **正确**
  - 补充：**存储结构未定**，可**顺序存储（数组）**

【考点：叶结点与度】

- **叶结点**：**向下无分支**，即**无孩子**
- **度**：**分支数量**
- 结论：**叶结点度为0** → 选 **B**

十二、二叉树 **02:33:48**

树的结构与基本概念

【核心考点：树的基本概念】

- **结点间关系**：**层次关系**（树最鲜明特点）
- **根节点层数**：**从第0层开始**（现行规定）
- **树的度**：所有结点度中的**最大值**
- **最少节点数**：**0个**（空树）

二叉树定义与性质

【核心考点：二叉树定义】

- 定义：**树 + 限制条件**（每个节点向下分支**最多2支**）
  - 注意：**最多**分2支，非必须分2支（可分0、1、2支）
  - 超过2支（3、4、5支）则**非二叉树**
- 扩展：限制改为最多3支→**三叉树**；最多4支→**四叉树**（不研究）

【核心考点：二叉树结构术语】

- **左右孩子**：节点最多分2支，自然分出左右位置
  - 例：a的左孩子=b，a的右孩子=c
- **左/右子树**：左/右孩子及其**所有后代节点**
- **节点度**：仅三种可能（**0**→叶节点，**1**，**2**）

【核心考点：二叉树性质（重要结论）】

- **第l层最多节点数**：**2^l个**（最多，非必须）
  - 第0层：最多1个（2^0）

- 第1层：最多2个（2^1）
- 第2层：最多4个（2^2）
- 规律：每层**乘以2**递增

- **总节点数上限**（高度为h）：**最多2^h - 1个**
  - 例：高度为3，最多7个节点（全满状态）

1.例题:二叉树度为二 02:37:55

我们来看一下题目啊，这个关于。二叉树的概念a选项。他说，二叉树的度为二，我们说不能超过二，但是不代表说一定是二，就是说如果它有。分出两支的就为二，但但是他不一定说必须分出两支来，对吧？所所以这个是错的啊b，他说可以小于二是对的吧，比如说这这样这样一个二叉树的度。就是这个小于二的啊，所以这个是对的c选项，他说至少有一个节点的度为二，也就是说他说必须有至少有一个节点要分两支出来，没有这种规定啊，就只要不超过就行了啊。然后第一他说任何一个节点的度都为二，显然不是的啊，再看这道题，它有四句话，就是说哪几句话是对的啊，我们看一下第一句话，他说只有一个节点的二叉树。只有一个节点的二叉树度为零，只有一个节点，那就根结点嘛，然后根结点它的向下没有分支，所以它度为零，所以这个这个是对的啊。第二句话，二叉树的度为二，这是显然错的。哎，我们说只要不能超过啊，但是不是说一定为他啊，然后呢，第三个二叉树左右子树可以任意交换。呃，很显然，比如说，比如说a。ABC如果任意交换的话，下面就变成了acb，这显然是两棵不同的树，对吧？所以交换的时候，它的意含义就不一样了，所以这个显然是错的啊。好，然后第四个呃，这个完全这里面有两个概念，完全二叉树和满二叉树，咱们后面再讲啊呃，当然这个题前面三句话已经判断出来，它已经可以选出来了，第一个是对的，二三都是错的，那就把。有二的都给它排除掉，那么就选a再看这道题，这道题他已经说了，跟着层号为一，如果他没说的话，我们就是为零啊，那么现在就是根是在第一呃层号一二。他问第五层一二三四五哈，第五层最多有多少好？那么第一层最多是层号一，那就是一个，那就是乘以二嘛，两个这是四个，这八个。这是16啊，所以他的现在他说的这个第五层就是16，所以选c。第五章的第三个考点，卡特兰数。这个卡特兰数这个概念的提出呢？它是为了解决这样一个问题啊，就是n个节点，它可以有多少种不同形状的二叉树？好当这个n取值比较小的时候，咱们可以直接画出来，但是N多了之后你就很难画出来，所以就得到了一个公式好，那么这个n个节点有多少种不同形状的二叉树，我们就叫做卡特兰数，它的公式是这样的啊，就是f。nn就是有多少个节点，它的公式是c。c2n。然后2n里面选n个，然后再除以n+1，这个c呢？就是概率里面选择从多少里里面多少个里面选多少个出来，这应该是高中知识啊，好，咱们来验证一下这个公式。你看比较少的时候，我们可以画出来啊，比如说当n=1的时候n=1的时候，那么实际上就只有一个根结点吧，那它的形态就只有一种嘛好，所以它得到的这个得到的这个。呃，你把这个一带到这个里面来，那么不就是c二二两个里面选一个，然后再是一+1吗？这个上面二两个里面选一个得到是二吧，然后下面是二二÷2是不是得到一啊？哎，所以这个公式是对的啊，好，接下来再验证一下，当这个n取二的时候n取二的时候就是有两个结点，那么两个结点它就实际上有两种形态，首先根结点肯定是有的，然后。向左边有一个位置嘛，因为他规定是二叉树啊，那就是这是一种形态，然后这是一种形态好，所以他的这个得到是二，那这个二你带进入算一下嘛，那就是。四个里面选两个，然后二二+1吧，好四个里面选两个算法呢，就是四×3，然后再除以二，再除以三。呃，你看这个约掉了二这个约掉了，所以刚好是等于二好那么假设现在有三个节点，那么三个节点，它的这个形态就比较多了啊。好，那么这个时候呢？首先根节点肯定是有的好，他可以向左来一个对不对？也可以向右来一个好，那么我可以向左来一个不向右来，再向左来一个对吧？好，然后向左来一个，再向右来一个，那么也可以怎么呢？向右来一个，再向向右来一个，再向右来一个，向

左来一个，就那么只有这五种哈，所以他的这个。不同的形态是五，那么我们把三代入到这个卡特兰数这个公式来算一下啊，那就c六三嘛c六三，然后除以三+1好c六三，那就是六×5×4，再除以三。乘以二再除以四好，那么这个得到你看啊，这个和这个约掉了吧，这个和这个约掉了是不是等于五啊？跟这个是匹配的啊，你看我们。我们这个节点取一二三的时候，我们还能画出来，但是当节点取四的时候，实际上也能画出来啊，但是就比较复杂了，那后面再多的话，那可能就画不出来了，那我们就直接用这个计算了，那就直接用这个公式进行计算了，好看一下这道题啊，

【考点：二叉树基本概念】

- **度定义**：节点分支数，二叉树度**不超过2**（非必须为2）
- 选项判断：
  - A：度为2 → **错误**（可为0、1、2）
  - B：可小于2 → **正确**
  - C：至少一个节点度为2 → **错误**（无此强制要求）
  - D：任何节点度均为2 → **错误**

【考点：二叉树性质判断】

- 语句1：**单节点二叉树度=0** → **正确**（仅根节点，无分支）
- 语句2：二叉树度为2 → **错误**（仅上限约束）
- 语句3：左右子树可任意交换 → **错误**（交换后结构不同，如ABC与ACB）
- 语句4：完全/满二叉树概念 → 后续章节
- 结论：仅语句1正确 → **选A**

【考点：二叉树层节点数计算】

- 层号约定：**根为第1层**（未说明则根为第0层）
- 第n层最多节点数：**2^(n-1)**
- 第5层最多：**2^4 = 16** → **选C**

【考点：卡特兰数（不同形态二叉树计数）】

- 定义：**n个节点可构成的不同形态二叉树数量**
- 公式：**f(n) = C(2n, n) / (n+1)**
- 验证：
  - n=1：C(2,1)/2 = 1 → **1种形态**
  - n=2：C(4,2)/3 = 2 → **2种形态**
  - n=3：C(6,3)/4 = 5 → **5种形态**
- 应用：n较大时**直接套公式计算**，避免枚举

2.例题:3个结点二叉树 02:42:31

第五章 第四个考点 满二叉树

【考点定位】

- 承接：**二叉树**加限制条件 → **满二叉树**
- 关联：**卡特兰数**（n=3时，不同二叉树形态数为**5**）
  - 节点少：可**直接画图**验证
  - 节点多（≥4）：**必须用公式计算**

【满二叉树定义】

- 核心限制：**每一层节点数均取最大值**（即“满”）
- 判定示例：
  - 给定二叉树：**每层均满** → 是满二叉树
  - 第二层缺1个节点 → **不是**满二叉树
  - 第三层缺节点 → **不是**满二叉树

【节点数规律】

- 递推关系：**第0层1个，每层×2**（1、2、4、8…）

- 总节点数：**由高度直接计算**
- 考法：**常结合其他题目综合考查**

十三、完全二叉树 **02:44:00**
第五章的第五个考点
【考点定位】
- 概念：**完全二叉树**（在二叉树基础上进一步限制）

【核心定义：两个限制条件】
- **结构限制**
- **除最后一层外，其余各层必须为满二叉树**
- 即：不看最后一层，整体即为满二叉树
- **排列限制**
- **最后一层节点必须从左到右依次排列**
- 禁止跳跃空缺

【判定示例】
- 满足条件示例：最后一层节点连续无空缺
- 反例：最后一层节点存在跳跃（如空缺后右移），则**不构成完全二叉树**

【考点：满二叉树与完全二叉树关系】
- **满二叉树一定是完全二叉树**
  - 依据：满二叉树不看最后一层仍为满二叉树；最后一层满，必满足从左到右依次排列
- **完全二叉树不一定是满二叉树**
  - 依据：完全二叉树最后一层仅要求依次排列，**不要求排满**

1.例题:完全二叉树叶子结点 02:45:38
我们来看一下这道题啊，他说一个高度为四，高度为四，就是一共有四层，但是从零第零层开始啊，就是零一。那么是零一二三，这就是高度为四，然后呢，他说第四层上有三个叶子节点，他说这个四层实际上就是层数为三啊。啊，就是层数三好，也就是说如果说你是这个层数，从一开始它就是第四层，因为这个改版之后它这个叫法有点不同啊，好第四层上有三个啊，也就是这里有三个三个叶子节点。那么，它的叶子节点的个数是多多少？那么，根据完全二叉树的我们的规定，那么前面的三层，它就一定是满的，对不对？好，所以前面三层一定是满的。好，所以前面三层一定是这个形态，那么最后一层呢？就是就是这样一个形态好，然后他现在问他这里面，他整个里面的叶子结点个数有多少，那么这一个这一个，然后这三个，所以是五个吧，所以这个选b再看一下这道题啊，他说有五个。分支节点好，什么叫分支节点？就是向下有分支的就叫分支节点，所以实际上就是除了叶节点都是分支节点啊，就是不管是分一支还是分两支，都是分支节点，但是没有分就不是分支节点。他说有五个分支节点的完全二叉树叶节点个数只能是多少？好，咱们可以试一下嘛好，那你看既然有五个分支，他首先有根吧，然后呢？他既然已经说了是完全二叉树，所以他前面几层。就肯定是满的啊，你就先画几层，大概画几层看一下啊，你看比如说我现在画了几层，这现在有几个分支节点呢呃，比如说我先画三层啊，先画三层。好那么呢，这个时候他是的，他是的，他是的，这个时候有三个分支节点吧，那么因为他要有五个就不够，那我就继续再画一下啊。那么，再画一个，再画一个，再画一个。再画一个好，那么这个时候分支节点有几个？他是的，他是的，他是的，他是的，他是的五个啊，这就五个，那就已经够了，已经满足我们这个条件了，所以呢，我们现在画的这种形态的完全二叉树就满足我们条件，那你看它的叶结点有多少个啊？这里有两个，这里有四个，就有六个，所以叶节点为六是满足条件的啊，那么我们看一下还有没有其他可能性，那么实际上这个叶节点可以不要，你看这个叶节点不要的情况下，它仍然也满足

我们的要求啊，你看这里不要。你看，也就是说这个它分一支，它仍然是分支节点，所以分支节点是五个没改变好，然后呢，它仍然是一个完全二叉树，它只要最后一排是从左到右排列，它没有跳就行了啊，所以这个时候的叶节点为五个也满足要求，所以。只能是五和六啊，所以这个选a好，他为什么不能是为什么不能是这个呃七个呢？比如说我这里。添一个是六个，如果说我这里再添一个的话，那么这个时候呢？这个时候分支结点就不是五个了，他又变成分支结点了，好，那就那就破坏了这个规则，所以只有可能是五和六。

【考点：完全二叉树叶子结点数计算】

- **高度为4**：层数从0计，即**0~3层**；“第四层”即**层数3**
- 第四层有**3个叶子结点**
- 完全二叉树性质：**前三层必为满二叉树**
- 叶子结点构成：
  - 第三层：**2个叶子**（第四层3个结点的父结点）
  - 第四层：**3个叶子**
  - 总计：**2 + 3 = 5个**
- 答案：**B**

【考点：分支结点数与叶子结点数关系】

- **分支结点**：**有子结点**（1个或2个）的结点；**叶子结点**：无子结点
- 条件：**5个分支结点**的完全二叉树
- 推导过程：
  - 画**3层满树**：分支结点**3个**（根+2个内部结点）
  - 需**5个**，继续扩展：增加**2个分支结点**
  - 形态一：末层**4个叶子** → 叶子总数 = **2 + 4 = 6**
  - 形态二：末层**3个叶子**（去掉最右叶子） → 叶子总数 = **2 + 3 = 5**
- 关键限制：**完全二叉树末层从左至右连续**，不可跳跃
- 排除7个：若再添1个叶子，其父结点变为**分支结点**，分支结点数变为**6**，破坏条件
- 结论：叶子结点数**只能为5或6**
- 答案：**A**

2.二叉树的结点的关系 02:48:22

第五章的第六个考点

【核心考点：二叉树节点按度分类及数量关系】

- **节点分类依据**：二叉树节点**度（向下分叉数）仅可为0、1、2**，不存在度为3的节点
- **分类表示**：设度为0、1、2的节点个数分别为 **n0、n1、n2**
- **核心结论**：**n0 = n2 + 1**（0度节点数 = 2度节点数 + 1）
  - 应用：可直接使用，无需推导

【结论推导（辅助理解）】

- **总度数 = 总边数**（每个分叉对应一条边）
- **总边数 = 节点总数 - 1**（除根节点外，每个节点均有一条边连接父节点）
- **等式建立**：**0×n0 + 1×n1 + 2×n2 = (n0 + n1 + n2) - 1**
- **化简结果**：**n2 = n0 - 1**，即 **n0 = n2 + 1**

1) 例题:二叉树叙述判断 02:51:45

我们来看一下题目啊，你看这个，他说他问下面四个选项，错误的是他是选错误的啊，说明有三个是对的，一个是错的。好，我们看一下a，他说二叉树可以为空，是可以为空啊，根节点也可以没有啊，所以这是对的，然后二叉树可以保存在这个数组里，就是二叉树呃，是这个逻辑结构吧？就是然后顺序存储，这也是可以的啊，然后看一下c选项，它说叶节点叶节点就是n0的n0的个数就是叶节点就是度数为零嘛，然后说多余度为呃，多余度为一的个数n0大于n1。好，这个我们我们刚才的结论只有n0与

n2之间的关系，没有这个关系，对吧？所以这个就是错的啊，再看一下呃d选项，他说叶节点个数，这是。业绩点个数就是n0，然后这个多于这个呃呃度为二，那就是n2，因为我们知道n 0=n二+1，所以他说。他说他比他多，因为他比他永远大一个嘛，所以他比他多是对的啊，所以这个题错的就选c再看这题啊，这我们就可以直接用结论，我们结论n 0=n二+1，然后这道题里面你看他说已经告诉了零的是六个，就是这个是六个。好，然后他问这个度为二是多少？那就这个就那么这就直接为五呗，所以这有五个再看这道题啊，他说有五个二叉树里面有五个度为二，也就是n2是等于五。然后呢，三个度为1n一是等于三，然后根据我们的结论，那个n0比n2多一个，所以n0就是等于五+1就是六，然后他问。共有多少个因为二叉树里面只有这三种，那就把这三个相加五+61，再加上三就14，再看这个题啊，那这个就是直接考结论了n 0=n二。n2+1，所以就直接选这个选d。第五章的第七个考点，二叉树的顺序存储呃，因为二叉树它是一种逻辑结构啊，就是它的存储结构没有确定，

【考点：二叉树性质判断题】

- 题目要求：**选错误项**（三对一错）
- A选项：**二叉树可为空** → 正确（根节点可无）
- B选项：**数组存储二叉树** → 正确（顺序存储可行）
- C选项：**叶节点数n0 > 度1节点数n1** → **错误**（仅存在n0与n2关系，无n0与n1关系）
- D选项：**叶节点数n0 > 度2节点数n2** → 正确（**n0 = n2 + 1**，恒多一个）

【考点：二叉树节点数计算】

- 核心结论：**n0 = n2 + 1**
- 题1：已知n0 = 6 → **n2 = 5**
- 题2：已知n2 = 5，n1 = 3 → n0 = 5 + 1 = 6 → **总节点数 = n0 + n1 + n2 = 6 + 3 + 5 = 14**
- 题3：直接套用结论 **n0 = n2 + 1** → 选D

【考点：二叉树的顺序存储】

- 二叉树为**逻辑结构**，存储结构未确定

十四、二叉树的顺序存储 02:53:32

二叉树的顺序存储

【存储工具与规则】

- 工具：**数组**
- 规则：**逐层存放**（第一层 → 第二层 → 第三层）

【分类：完全二叉树】

- 特点：**前面若干层满，最后一层从左到右连续**
- 存储方式：按层直接连续存放，**无空缺**
- 示例：ABC DEF → 数组排列为 **ABCDEF**

【分类：非完全二叉树】

- 处理方式：**仍按完全二叉树结构看待**
- 缺失节点：**对应数组位置留空**
- 示例：第二层缺一节点 → 数组对应位置空；第三层缺两节点 → 连续空两位
- 核心：**按层编号，缺位补空**

1.例题:二叉树顺序存储 02:55:09

二叉树顺序存储例题解析

【题目信息】

- 题干：**20个节点**的二叉树，采用**顺序存储**
- 存储方式：**一维数组**，按层存放
- 关键前提：顺序存储时**无论原二叉树形态，均按完全二叉树处理**（前面若干层必为满层）

【数组定位】

- 数组起始下标：**b0**
- 节点位置：**b7** = 数组**第8个**元素
- 根节点位置：**b0**
- 已知条件：**b7处保存节点v**

【求解目标】

- 求：节点v的**父节点**在数组中的存储位置

【解题步骤】 - 按完全二叉树层序排列，定位b7对应节点

- 其父节点位于**上一层**，对应数组位置为**b3 -** 核对计数：b0→b1→b2→**b3**（父节点）

【答案】

- 父节点存储于 **b3**
- 正确选项：**C**

十五、二叉链表 **02:56:06**

第五章的第八课考点二叉链表

【核心考点：二叉链表定义与结构】

- **二叉链表**：二叉树链式存储方式
- 节点结构：**三域**（左指针域、数据域、右指针域）
- 指针指向：左指针域 → **左孩子**；右指针域 → **右孩子**

【结构示例：二叉链表转换】

- 根节点 a：左指针 → b，右指针 → c
- 节点 c：**左右指针均为空**（无孩子）
- 节点 b：左指针 → d，右指针 → e
- 节点 d、e：**左右指针均为空**

【核心考点：指针域数量分析】

- 总指针域数：**2n**（n个节点 × 左右2个指针域）
- 非空指针域数：**n-1**（对应n个节点二叉树的**n-1条边**）
- 空指针域数：**n+1**（2n - (n-1) = n+1）

1.例题:二叉树指针域 **02:58:12**

二叉树存储结构考点解析

【核心考点：二叉链表存储结构】

- **存储方式**：二叉链表
- **指针域总数**：**2n**（n个节点 × 每个节点2个指针域）
- **非空指针域**：**n-1**（n个节点的树有n-1条边）
- **空指针域**：**n+1**（2n - (n-1) = n+1）
- **结论**：选项D正确

十六、二叉树的遍历 **02:58:34**

第五章的第九个考点，二叉树的遍历好，这个遍历呢，也是一个我们新的概念啊，那么什么叫遍历呢？遍历就是说，通过某种次序，把所有的节点访问一遍啊，就比如说假设有十个节点，我通过某种次序，把这十个节点访问访问一遍，那么这个遍历啊，实际上在生活中最常见的例子呢，就是点名。就比如说一个班上点名或者一个队伍点名啊，就假设说呃，你看比如说一个班上有30个学生，那么点名实际上就是按照某种次序把这个学生都访问一遍吗？好，那你你可以想象一下啊，那我在这个给学生点名的过程中，我大概要遵循两个原则，第一个就是什么呢？第一个就是我不能漏，比如说我点30个人，我不能点29个。哎，点28个，那就漏一个，漏两个不能漏，就是每个都要点到，这是第一个啊呃呃，说到那说回到我们这个访问这个节点，就是说你访问

这个节点你不能漏，你不能说十个节点，你只访问九个哈。那么，在给学生点名的时候，呃，那我们如果一个学生已经点过了，我们就不能再点，对吧？就不能重复的点，比如说你喊这个点，这个小名已经点过一次了。那么点过一次，你就不要再点一次了，那么同样那我们这个对这个呃二叉树的遍历就是访问节点的时候，那我们就是一个节点，只能访问一次，就不能说访先访问他了一次了，然后又访问一次，那就不行。那么，经过这样分析啊，那我们就可以把这个二叉树的遍历呃，再简单说一下，就是再简单总结一下意思，就说我通过某种次序把所有的节点都点一遍，这所有的节点不能漏，并且呢，每个节点我只能点一次。那么接下来呢，又产生了一个问题，那我们刚才说的某种次序，那我到底按照什么次序来呢？呃，那这个次序我们常用的次序就有四种啊，这四种呢分别叫做？分别叫做先序，中序，后序和层序啊，那我们来分析一下这四种，我们先用比较简单例子来分析，我们后面再讲比较呃复杂的，那我们这边先给一个比较简单的二叉树啊ABC这一个比较简单，二叉树那么什么叫先？先序呢？先序就是我按照根左右的这个次序来访问，这个根就是指根结点哎，左是指左子树，右是指右子树好，那我们来看一下啊，我们给的这个二叉树按照根左右，那就是。那就是先访问根嘛，就是a，然后左子数这个时候左子数只有一个，那就是b嘛好，然后右子数只有一个，那就是c好，所以我们按照先序得到的这个遍历序列就是ABC。好，那么第二种中序中序是按照左跟右这个次序来访问，也就是呢，我先访问根的这个左指数，然后再访问根，再访问根的右指数，那么对于我们给这个二叉树。那么根是aaa的，这个左子数就就是b，所以是先访问b再访问根a好，最后访问根的这个右子数，那就是c。好，接下来第三种叫做后续，后续就是按照左右根的这个次序来访问左就是根的左子数，然后呢，再访问根的右子数，最后访问根好，那么对于我们给的这个例子，那就先访问。b再访问c再访问a好最后一种呢，比较容易理解，叫做程序程序呢，就是先访问第一层，再访问第二层，那么这是第一层，这是第二层好，所以这样我们就得到了。这个ABC那么这个考点呢？我们主要是介绍一下这个二叉树遍历的概念，然后呢，我们给的这个比较简单的例子，我们为什么写出来比较快呢？因为我们给的这个例子很简单，如果说他给的比较复杂的话，写出来就没那么容易。啊，那么如果给的比较复杂的时候，我们就需要用特定的方法，那么特定的方法，我们下一个考点来介绍。

【遍历核心概念】

- **定义**：按**某种次序**访问所有节点
- 生活类比：**点名**（按序访问每个学生）
- **两大原则**：
  - **不漏**：所有节点必须全部访问
  - **不重**：每个节点仅访问一次

【四种遍历次序】

- **先序**：**根 → 左子树 → 右子树**
- **中序**：**左子树 → 根 → 右子树**
- **后序**：**左子树 → 右子树 → 根**
- **层序**：**按层**（第一层 → 第二层）依次访问

【简单示例：二叉树ABC】

- 先序序列：**A B C**
- 中序序列：**B A C**
- 后序序列：**B C A**
- 层序序列：**A B C**

【考点定位】

- 本考点：**遍历概念**与**简单序列推导**
- 复杂二叉树遍历方法：**下一考点**详解

1.二叉树遍历方法 03:01:59

第五章的第十个考点二叉树遍历方法

【考情与定位】

- 核心问题：**复杂二叉树遍历次序难写**
- 程序遍历（按层输出）：**无需方法，直接按层输出**
- 研究对象：**先序、中序、后序**三种遍历
- 核心方法：**整体法**、**扩展法**

【方法一：整体法】

- 核心思想：**将根与左右子树各视为整体**，逐步缩小范围至子树内部
- 操作步骤：
  - 根节点固定位置（依遍历次序）
  - 左右子树各视为整体，确定其相对位置区间
  - 缩小至子树内部，重复上述过程
- **先序遍历（根左右）**
- 根A固定首位
- 左子树BD整体位于第2、3位；右子树CE整体位于第4、5位
- 缩小至BD：根左右 → **B空D** → 得 **BD**
- 缩小至CE：根左右 → **C空E** → 得 **CE**
- 结果：**A BD CE**
- **中序遍历（左根右）**
- 根A固定中间
- 左子树BD整体在前；右子树CE整体在后
- 缩小至BD：左根右 → 空B D → 得 **BD**
- 缩小至CE：左根右 → E C空 → 得 **EC**
- 结果：**BD A EC**
- **后序遍历（左右根）**
- 根A固定末位
- 左子树BD整体在前两位；右子树CE整体在中间两位
- 缩小至BD：左右根 → 空D B → 得 **DB**
- 缩小至CE：左右根 → E空C → 得 **EC**
- 结果：**DB EC A**

【方法二：扩展法】

- 核心思想：**从根节点（第0层）开始，每次向下扩展一层**，逐层合并遍历结果
- 操作步骤：
  - 初始仅根节点
  - 向下扩展一层，将新扩展出的子树按对应遍历次序合并
  - 重复扩展直至覆盖全部节点
- **先序遍历（根左右）**
- 第0层：A
- 扩展第1层：A扩展出BC → 根左右 → **ABC**
- 扩展第2层：B扩展出D，C扩展出E → 分别按根左右合并 → **A BD CE**
- 结果：**A BD CE**（与整体法一致）
- **中序遍历（左根右）**
- 第0层：A
- 扩展第1层：A扩展出BC → 左根右 → **BAC**
- 扩展第2层：B扩展出D，C扩展出E → 分别按左根右合并 → **BD A EC**
- 结果：**BD A EC**（与整体法一致）
- **后序遍历（左右根）**

- 第0层：A
- 扩展第1层：A扩展出BC → 左右根 → **BCA**
- 扩展第2层：B扩展出D，C扩展出E → 分别按左右根合并 → **DB EC A**
- 结果：**DB EC A**（与整体法一致）

【方法对比】

方法	**核心思路**	**适用特点**
整体法	根与左右子树各视为整体，逐步缩小范围	从整体到局部，逻辑清晰
扩展法	从根开始逐层向下扩展，逐层合并	从局部到整体，逐层推进

【核心结论】

- 两种方法结果**完全一致**，可互为验证
- 复杂二叉树遍历：**按此方法逐步操作即可**

1) 例题:二叉树遍历 03:08:33

好，我们来看一下这道题啊，这道题呢，它就它就是说给了一个二叉树，它问下面哪个四种这个？呃，便利次序哪个能得到这个abcd？实际上它就是考你把这四个次字四种便利方方法写呃，四种便利的次序写出来啊。呃，当然，他给这个也比较简单呃，但是咱们主要是掌握操作方法啊，不是并不是说一定要操作非常复杂的，首先我们说这个层序层次遍历就是程序吧，就是这个永远是最简单的，那就先第一层，再第二层，再第三层吧，所以这个这个得到的是。abcd，然后这个得到的刚好就和我们想要得到是一样的，所以这个题就选d啊好，然后我们看一下其他几个为什么不对啊？好，那么看一下a先序，那么这个先序哎，如果说你用这个整体法的话啊呃，那么就是跟跟左把这个看成一个整体吧，跟左右哈，那么就是a。呃BD两个次序，不知道c，然后BD再看这个呃跟左右那就是b空d，所以就是b。d那么b选项的这个中序用整体法的话，那是左跟右啊，那么就是BD呃BD两个在前，然后AC，然后单独看BD的话，然后又是。左跟右就是空BD吧，所以就得到BD好，然后这个后续的话，那么就是左右跟好，那么左在前BD在前，然后右。根然后单独看这个BD的话，然后在左右跟那就是空db空db好那么那么发现前面三个和我们要的不一样啊，那么同样呢，我们也可以用扩展法写一下啊，那么这样你就可以把它看成三层。好，那么先写这个先序，那么先看前两层的话，先序的话，那么就是呃，那么就是跟左右，所以就是ABC　ABC，然后b扩展出了一个d吧。然后b扩展d也是这个根左右，所以就是a然后呃b空db空DC那么中序，先看前面两排的话，那么中序的话是这个。啊左跟右那么就是bac，然后这个时候b扩展出来一个d的话，哎，那么b和d再用这个左跟右就是空BD啊。那所以就得到bdac好，也就是b扩展出了BD好，再看这个后续的话，后续看前。前两排的话，那么就是bca，然后呢b扩展出了一个d，那么单独看BD的话就是空。db，好，那你们就是dbca，也就是b扩展出来一个db。

【题型定位】

- **二叉树遍历次序**辨析题
- 目标：判断四种遍历结果是否等于 **abcd**
- 核心：掌握**操作方法**（整体法/扩展法）

【层次遍历】

- 规则：**逐层从左至右**输出
- 结果：**abcd**
- 结论：**与目标一致，本题选D**

【先序遍历（整体法）】

- 规则：**根左右**
- 过程：a → (b空d) → c
- 结果：**abdc**

- 与目标不符

【中序遍历（整体法）】

- 规则：**左根右**
- 过程：(空bd) → a → c
- 结果：**bdac**
- 与目标不符

【后序遍历（整体法）】

- 规则：**左右根**
- 过程：(空db) → c → a
- 结果：**dbca**
- 与目标不符

【扩展法验证】

- 适用场景：**三层结构**二叉树
- 先序：前两层 **abc**，b扩展d → **abdc**
- 中序：前两层 **bac**，b扩展d → **bdac**
- 后序：前两层 **bca**，b扩展d → **dbca**
- 结论：**三种方法均无法得到abcd**

十七、遍历确定二叉树 **03:10:56**

第五章的第11个考点

【核心结论】

- **已知遍历序列，反推二叉树**：仅两种组合可行
  - **先序 + 中序**
  - **中序 + 后序**
- **关键规律**：**必须包含中序**（中序分别搭配先序或后序之一即可）
- 推导过程：无需掌握，直接记忆结论

【反推原理与操作思路】

- **根结点定位**：
  - 先序序列：**第一个元素为根**
  - 后序序列：**最后一个元素为根**
- **左右子树划分**：
  - 将已定位的根结点代入中序序列
  - 根结点在中序中作为**分界线**，左侧为左子树，右侧为右子树
- **迭代操作**：对划分出的左右子树**重复上述步骤**，直至完成整棵二叉树构建

1.例题:二叉树后序遍历 03:12:17

我们接下来直接看题目好，首先看这个题目啊。这个就是直接用我们的结论，我们说中序加上任意的前序和后序，任意一个都行啊，所以必须要有中序，所以这里必须要有中序，所以这个是对的，所以选c再看一下这道题，它告诉了前序。和中序，然后他问后序，那我们的做法是前序和后序呃，前序和中序可以先把二叉树确定出来，然后再通过二叉树把它的后序写出来啊，好，那我们就。那我们就看一下啊，我们通过前面的两个怎么样把这个二叉树写出来那么呢？根据前序的特点，那么前序的第一个就是根对不对？所以这里我们就确定a是根。好，然后a分左右好，然后这个时候你把a带入到这个中序里来看，那么这个a相当于是是一个分界线，那么c cd b都在它的右边，那就说明什么？说明这个cdb都在右边，那么左边就没有啊，也就是说我确定这三个都在右边，只是我不知道它怎么排列的，那么这个时候呢，我再回到前面这个前序里面看，那么既然a已经确定了a，我就不用看了，那我就单独看bcd，那么bcdb在最前面。那就说明，对于bcd来讲b是根对不对好，所以那就说明b是这里的根好，然后呢，再把

这个b带回到这个后面的这个里面来看，那你看啊，那么也就是我只看这个bcd的话，那么b是一个分界线。CD都在b的左边，那就说明c和d都在d的呃c和d都在b的左边，只是我不知道c和d是怎么排列的，那么接下来我再回到前面的前序啊，那么AB都确定了这两个就不用看了，就单独看CD。那么cdc在前面那么那就说明对于c和d来讲c是根好，所以那就说明这里是c好，然后再继续回到后面来看后面来看，这个时候AB就不用看了，单独看CD那么以c为分界线d在c的右边。那就说明第一。第一，一定是它的右边，所以这样我就这样，我就把它写出来了啊，那这个二叉树实际上就是这样，就把它写出来了，那么二叉树已经知道了，那我要写它的后续啊，那我们就采用这个整体法，把这首先把这边看成整体吧。这边看成整体好，然后后续那就是呃左右跟那么说明什么a1定在最后啊好，然后再单独看这个这个这个里面这个整体那么这个单独看这个里面整体呢，这个时候。我又可以进一步的把这个看成一个整体，对不对？好，那么这个时候左右根那么就是b在最后好，然后再单独看这个。单独看CD　　CD左右跟的话，那就是空DC，那就是空d空DC，那就是dcba，

【考点：遍历序列确定二叉树】

- **核心结论**：**中序 + 前序/后序任一**，可唯一确定二叉树
- 本题判定：**必须含中序**，故正确选项为 **C**

【考点：前序+中序还原二叉树】

- **还原原理**
- **前序首元素 = 根节点**
- **中序根节点为界**：左侧为左子树，右侧为右子树
- 递归重复：**前序定根，中序分左右**
- **本题还原步骤**
- 前序首元素 **a** → 根
- 中序中 **a** 右侧含 **c、d、b** → 均在右子树
- 前序剩余 **b、c、d**，**b** 在前 → **b** 为右子树根
- 中序中 **b** 左侧含 **c、d** → 均在 **b** 左子树
- 前序剩余 **c、d**，**c** 在前 → **c** 为根
- 中序中 **c** 右侧为 **d** → **d** 为 **c** 右孩子
- 还原结果：**a 为根，右孩子 b，b 左孩子 c，c 右孩子 d**

【考点：后序遍历（整体法）】

- 后序规则：**左右根**
- 整体法：**a 必在最后**
- 子树递归：**b 在子树最后**，**c、d 按左右根排列**
- 最终后序序列：**d c b a**

2.例题:二叉树后序遍历 03:14:41

那就是个选d，再来看一下这道题啊，那么这道题的节点就多一点，但是操作方法是一样的啊，你慢慢做是一样的，他告诉了先序和中序。好，然后问后续那我们同样是先通过先序中序把二区二叉树画出来，然后再遍历他的后呃，再写他的后序啊好，那我们看啊，我们首先在先序里面找它的根结点，它第一个就是a吧。所以就说明这个a是这个根结点好，然后再把这个a代入到这边来看，那这个a就相当于是一个分界线，那就说明这三个在左边。这四个在右边，只是说它的排序我不知道，但是我知道它它的它，它是在左在右啊，然后我们把这前面这三个带回到前面来看啊，那么这个时候a确定就不用看了，那么这个时候就看这三个。那就看这三个，那么这三个是b是在最前面说明，对于这三个b是根结点好，所以就说明左边这三个。b是根结点，同样把这这后面的这一四个拿到这里来看。c在最前面，那就说明c是右边的根结点，然后呢，这个时候把这个b和c带回到后面来看啊，那么你看啊。那么单独看这前面那么那么那么以b

为分界线，那就说明什么d和gd和g。都在它的左边，那么以这个c为分界线，那就说明什么？说明这个e在它的左边，然后hf在右边啊，也就是我能分出左右，但是不知道它具体的排序啊。好，然后再又拿回到前面来看啊，那么前面你看这些这些确定的，我们就不用看了啊，我们就直接关注这剩下的，你看剩下的d和g那么d在g的前面，那就说明对于dg来讲的话d是根结点。好，所以。d在这里，然后再把这个d拿回到这个中序来看，那么以d为分界线g在它的这个右边，那就说明g在这里，然后呢，刚才这个e这边因为只有一个e，所以e就必定在这里啊。只有一个，那就不存在这个排序的问题了哈，所以一也确定了，那这个一就不用看了，然后我们单独看这个fh那么f在h的前面说明。对于他们两个来讲的话f是根结点对吧？所以叫f在这里，然后然后再回到这里来看这个hf好，那么以hf为分界线的话h在。f的左边那就说明哎，说明这个h应该是在左边好，那么这样我们就把它这个二叉树，这样就把它这个二叉树画出来了，那么现在已经知道了二叉树要写它的后续啊。呃，因为这是一个选择题，所以呢呃，你可以观察一下它的选项，你也不一定要把它全部写出来啊，那么这样我用整体法，我把这看成一个整体，左边看先把左边看成整体，然后右边看成整体。好，那么后续的话，那就是那就是左右跟好，首先a肯定是在最后的n1a1定在最后，所以这两个直接排除掉了啊，然后呢，如果我们用整体法的话，那就说明左边这个整体它一定是前三个对不对，所以前三个一定一定是b。ddg，当然，具体的次数不知道呃，次序不知道，但是一定是这三个，那么你看这前面三个就不可能出现h吧啊，你看这三个。这三个。哎，就刚好是刚好是这三个嘛，好，所以这个一定是a对啊，所以呢，这个题其实不需要那么复杂，也可以直接把它选出来，我们刚才主要是为了这个呃，训练一下这个通过先序和中序把二叉树画出来啊。那么，这个题实际上不画二叉树，是不是也可以做出来啊？你看啊，那么先序，那就那就说明a是根结点啊，那么a，然后a通过这个分界线的话，说明这边是。根的这个左子数这边是根的右子数好，然后后续的话，那么后续的话a1定是在最后对吧好，所以这两个先排除掉。好，然后呢？呃，后续呢？把前面看成整体的话，那么这个一定是先访问这个左子数的三个，所以呢dgb　1定是最先的三个，当然它的次序不知道啊，所以这里是不能出现h的，所以这个也排除掉了，所以这个。直接选a也就是说可以不把二叉树画出来。第五章的第12个考点堆好。首先呢，堆这个概念啊，

【题型定位】

- **已知先序+中序**，求后序
- 核心方法：**先还原二叉树，再遍历后序**

【解题步骤：还原二叉树】

- **确定根结点**
- 先序首元素为根：**a为根**
- 中序以a为界：**左3个（d/g/b），右4个（e/h/f/c）**
- **递归确定子树根**
- 左子树（b/d/g）：先序中**b最前 → b为左子树根**
- 右子树（c/e/f/h）：先序中**c最前 → c为右子树根**
- **继续划分左右**
- 以b为界：**d、g均在b左**
- 以c为界：**e在c左，h、f在c右**
- **确定叶子与内部结点**
- 左子树剩余（d/g）：先序**d在前 → d为根**；中序以d为界 → **g在d右**
- 右子树剩余（e/f/h）：e唯一 → **e确定**；f/h中**f在前 → f为根**；中序以f为界 → **h在f左**

【后序遍历与选项排除】

- 后序顺序：**左右根 → a必在最后** → 排除非a结尾项
- **整体法**：左子树整体（b/d/g）必为后序前3个 → **前3个不可能出现h** → 排除含h项

- 结论：**直接选a**，无需完整画出二叉树

【考点延伸】

- 核心能力：**先序+中序还原二叉树**
- 技巧：**选择题可用整体法快速排除**，不必全图遍历

十八、堆 03:18:30

堆的定义与分类

【堆的基本概念】

- **堆**：**完全二叉树**（概念前已述，不赘述）
- 分类：
  - **大根堆**（别名：**最大堆**）
  - **小根堆**（别名：**最小堆**）

【堆的规则】

- **大根堆**：**任意父亲 ≥ 左右孩子**（任意节点值不小于其分出的两个子节点值）
- **小根堆**：**任意父亲 ≤ 左右孩子**（任意节点值不大于其分出的两个子节点值）

【实例验证】

- 右侧堆：**大根堆**（如5>左右孩子，4>左孩子，6>左右孩子，均不违反规则）
- 左侧堆：**小根堆**（如1<左右孩子，2<左右孩子，3<左孩子）

【判断方法】

- 给定序列判断是否为堆：
  - **按层铺满**（最后一层从左到右）构建**完全二叉树 -** 依据**大根堆/小根堆规则**逐节点检查是否违反

1.例题:堆的建立 03:20:08

第五章的第13个考点堆的建立

【题型定位】

- 任务：**判定最大堆（大根堆）**
- 方法：**还原完全二叉树** → 逐结点核验堆序规则

【选项A核查】

- 序列：75, 65, 30, 15, 25, 45, 20, 10
- 二叉树逐层铺满
- 核验：
  - 75 > 65、30 ☑
  - 65 > 15、25 ☑
  - 30 > 45 ❌（**违反堆序**）
- 结论：**非最大堆**

【选项B核查】

- 序列：75, 65, 45, 10, 30, 25, 20, 15
- 核验：
  - 75 > 65、45 ☑
  - 65 > 10、30 ☑
  - 45 > 25、20 ☑
  - 10 > 15 ❌（**违反堆序**）
- 结论：**非最大堆**

【选项C核查】

- 序列：75, 45, 65, 30, 15, 20, 25, 10
- 核验：
  - 75 > 45、65 ☑

- 45 > 30、15 ☑
- 65 > 20、25 ☑
- 30 > 10 ☑

- 结论：**满足最大堆全部条件**

【选项D核查】

- 序列：75, 45, 65, 10, 25, 30, 20, 15
- 核验：
  - 75 > 45、65 ☑
  - 45 > 10、25 ☑
  - 65 > 30、20 ☑
  - 10 > 15 ❌（**违反堆序**）
- 结论：**非最大堆**

【最终判定】

- 违反规则选项：**A、B、D**
- 正确选项：**C**

十九、堆的建立 **03:22:22**

那么现在给你一个序列，让你把它建成堆啊，那么怎么样操作啊？呃，那么我们这个因为我们知道堆有两种，所以它实际上是两种操作啊，但是它的操作的呃方法实际上都是一样的，它只是里面。调整的调整的方法不一样，那么我们直接用一个例子来实际操作一下啊，演示一下我们给的这三个规则啊，我们首先给了这个序列，他说建堆，我们先建大根堆啊。好，首先第一步就是把这个给的这个序列啊，就是给的这个序列，我先把它呃按层给它呃填到这个完全二叉树里面啊，所以那就是70。那么就是71，13，65。24。56，然后48。四十八九十二。86好，这这个就是完成我们的这个规则第一步啊，就是先把它排成一个完全数完全二叉树，然后呢，我们从最下面开始往上面调整啊，这个顺序不要搞错了好。那么现在因为我们要建立大建立大跟堆，那我们就简单来说就是要上大下小嘛，好，所以我们就先调整，那你看啊，86和24这个时候违反大跟堆的规则对不对？因为大跟堆我们要。父亲比他大嘛，所以我就把这两个交换一下啊，因为违反了，既然违反，我就跟他调整一下嘛，那么那么就把86调整到这里。然后24给他调整下来好，然后再继续继续往上调整，然后再看再看这一部分啊，就看。哎，看这一部分。好，那么看这一部分的话，那么那么是不是违反上大下小啊？那也违反，那我就把86和13。交换一下，然后你可能会说，那为什么我们不把56和它交换呢？因为你要选下面最大的给它交换嘛，因为你把56选上去了之后，这个86还是比56大，对吧？所以你就选下面大的和它交换。那86和13交换一下。好，13这是86好，同样我们再把这一部分也调整一下啊，看一下呃，这一部分呢，92和92也比这个65大吧，所以这里也要调整一下，所以92和65互换一下。所以这里是92，这里是65好，然后再看上面的三个，那么上面的三个92比70大吧，所以这里也要。交换一下。九十二七十好，那么这个时候已经完成了向下往上的调整，因为因为我们已经调整到最上面了，注意这个时候还没完啊，因为我们在调整的过程中从下往上调整过程中有些改动，所以我们还要从上往下再调整一遍啊。好，首先最上面的三个已经是好的啊，然后再继续观察，继续观察，你看哪里有没有违反的好，那么这里这个比他两个都大，对吧啊？这个比这两个都大好，那么。这里有一个问题是这里就是这个。不比它大好，所以就把这两个交换一下好，所以这个13和24交换一下。二十四一十三好，那么这样就调整好了，这样就就是一个这样就建立出来这个呃初始大根堆，那么同样还是这个序列，我们再建立一个小根堆啊，那么首先还是先把它填进去。首先填成一个完全二叉树，65，24。56。48。92。86好，然后从下往上减呃进行调整，那么这个时候是就是因为要调整为这个小跟堆，所以就是上小下大啊，那么你看啊。呃，

首先。首先，这里上小下大，这是呃满足的啊，然后这上小下大也满足好，然后这里就不满足啊，那么这个小的就和和这个交换一下啊。好，所以这里就。65和这个48交换一下。六十五四十八好，再看最上面最上面的那么13和70就要交换一下啊，好13和70交换一下。七十一十三，然后接下来从上往下检查一下啊，这个是上小下大好，然后呢，这里就不行啊，这里。这里就没有满足上小下大好，那我们就把小的调整下来，那就把24和70交换一下，24和70交换一下。其实。这里是24好，然后再看。看这边这边是上小上小下大好也是满足的，然后再看最后啊，这里这里上小下大也是满足的好，那么实际上这样就已经调整好了啊，所以这就是一个初始的。建立了一个初始的小跟队。第五章的第14个考点堆的插入好，那么对一个建好的大根堆或者小根堆，我们插入元素怎么操作啊？好，那我们的操作的规则是这样的，我们插入这个元素就排到这个二完全二叉树的最后一个啊，就是呃，最后一个位置。嗯，新被就新增加一个节点好，然后呢，插入之后再根据大跟堆和小跟堆的规则进行调整好，

【建堆操作核心流程】

- **核心规则**：堆分**大根堆**（上大下小）与**小根堆**（上小下大），操作流程一致，仅调整方向不同。
- **第一步**：将给定序列**按层填充**为**完全二叉树**。
- **第二步**：**自下而上**逐层调整（顺序不可错）。
- **第三步**：调整至根节点后，**必须再自上而下**复查调整（因自下而上调整可能破坏上层结构）。

【大根堆构建实例】

- 初始序列：70, 71, 13, 65, 24, 56, 48, 92, 86
- 调整过程：
  - **自下而上**：
    - 86与24交换（86>24）
    - 86与13交换（选**子树最大者**交换，非56）
    - 92与65交换（92>65）
    - 92与70交换（92>70）
  - **自上而下复查**：
    - 13与24交换（24>13）
- 结果：**初始大根堆建立完成**

【小根堆构建实例】

- 初始序列：70, 71, 13, 65, 24, 56, 48, 92, 86
- 调整过程：
  - **自下而上**：
    - 48与65交换（48<65）
    - 13与70交换（13<70）
  - **自上而下复查**：
    - 24与70交换（24<70）
- 结果：**初始小根堆建立完成**

【考点：堆的插入】

- 操作规则：新元素**插入完全二叉树末尾**（新增节点）
- 插入后：**根据大/小根堆规则**重新调整

第五章的第14个考点堆的插入好，那么对一个建好的大根堆或者小根堆，我们插入元素怎么操作啊？好，那我们的操作的规则是这样的，我们插入这个元素就排到这个二完全二叉树的最后一个啊，就是呃，最后一个位置。嗯，新被就新增加一个节点好，然后呢，插入之后再根据大跟堆和小跟堆的规则进行调整好，

【考点：堆的插入操作】

- **插入位置**：新元素置于**完全二叉树末尾**（新增节点）

- **调整规则**：插入后，依据**大根堆**（上大下小）或**小根堆**（上小下大）规则，**自下而上**调整至满足堆性质

1.例题:堆的插入 03:27:23

堆的插入操作示例

【核心考点：大根堆插入】

- **插入位置**：末尾（维持**完全二叉树**结构）
- **调整方向**：**自下而上**逐层比较
- **调整依据**：违反**上大下小**则交换
- **示例过程**：
  - 原堆插入 **40** → 置于末尾
  - 40 与父节点 **24** 比较 → **违反上大下小** → 交换
  - 继续向上检查 → 满足条件 → **调整结束**

【核心考点：小根堆插入】

- **插入位置**：末尾（维持**完全二叉树**结构）
- **调整方向**：**自下而上**逐层比较
- **调整依据**：违反**上小下大**则交换
- **示例过程**：
  - 原堆插入 **40** → 置于末尾
  - 40 与父节点 **70** 比较 → **违反上小下大** → 交换
  - 继续向上检查 → 满足条件 → **调整结束**

【易混淆点】

- 大根堆 vs 小根堆：**比较方向相反**（上大下小 vs 上小下大）
- 插入操作统一：**末尾插入 + 自下而上调整**

2.例题:堆的插入 03:28:34

好，我们来看一下题目啊，这个是在前面题目给了一个大跟堆，然后呢，我已经写在下面了，然后它是要插入一个新元素55，然后又得到一个。新的大根堆好，那我们来按照刚才规则做一下，那么插入的我就接在这最后面，所以55接在这里哈，然后再调整。调整，看有没有违反上呃，违反上大呃上大下小，那么这里就要交换一下吧，就是55。呃，55和30就交换一下，然后呢？这个55和45也交换一下吧。好，就是55在这里，45在这里好，然后再看这边这边呃，这边是符合的啊，然后再看上面，上面也是上面也是符合的好，然后再从上往下检查一下，这是上大。下小上大，下小上大，下小上大，下小好，所以这样就已经调整好了，调整好了，然后输出的话就按照按照行输出就行了，所以就是这样的。按照这个序列输出，所以就75，55，65，75，55，65，那就以这个开头对吧？其他几个都不对，所以这个选a。第五章的第15个考点堆的删除啊，就是删除堆的元素，然后呢，我们只要求这个掌握这个删除堆顶啊。好，然后还是分成两种，就是大跟堆删除堆顶和小跟堆删除堆顶好它的步骤，我写出来了啊，

【考点：堆的插入操作（大根堆）】

- **插入位置**：新元素55接在堆末尾
- **调整规则**：检查是否违反**上大下小**原则
- **调整过程**：
  - 55与30交换
  - 55与45交换
  - 最终位置：55在顶层，45在次层
- **最终检查**：自上而下逐层验证，均满足**上大下小**
- **输出方式**：按行输出序列

【题目解析】

- 插入后序列：**75，55，65，…**
- 选项判断：以该序列开头者为正确答案，选**A**

【考点：堆的删除操作】

- 范围：**删除堆顶元素**
- 分类：**大根堆删除堆顶**、**小根堆删除堆顶**
- 步骤：已给出，需掌握

3.例题:堆的删除 03:29:56

然后我们用这个一个实例看一下。这是一个小跟堆，我们之前建立的一个小跟堆，我们现在要删除堆顶，也就是删除这个13好，那我先把它空出来，好空出来，然后这个时候我用最后一个元素把它填上去啊，也就是我把这个86给它填上去。好，就把86填上去，填到这里来好，然后我再调整，这个时候调整，我们就不需要从下往上调整了，因为呢，我们直接把最后一个拿上去，下面都是排好的，我们就直接从上面往下调整就可以了哈。然后我们看从上到下哎，是否违反这个小跟堆嘛？就是违反上小下大哈，那么很显然是违反的好，所以那我们就把这个24和86交换一下。86。24好，然后继续检查啊，违反上小下大，那就56和86交换一下。好，然后再看这边呃，这边是不违反的哈，那么到这样就已经调整好了，那么我们再看一下这个大跟堆删除堆顶好，这也是我们之前给出的一个大跟堆好，那么删除堆顶先把这个92空出来，然后把最后一个这个13给它调整到上面。啊好，所以就变成这样了，然后从上往下检查好是不是违反上大下小，那么违反那么这个地方呢？就把这个86和这个13换一下。所以13在这里。86在这里好，然后再看啊，是否违反嗯，那么。那么，这里就把56和13交换一下。13。56好，这边不违反，那么这也已经调整好了，

【小根堆删除堆顶实例】

- **初始状态**：小根堆，堆顶为 **13**
- **删除步骤**：
  - **空出堆顶**：删除13，位置留空
  - **末元素上移**：将 **86** 填入堆顶
  - **调整方向**：**仅需自上而下调整**（下方已有序）
  - **调整过程**：
    - 检查 **上小下大** 性质 → 违反
    - **24 与 86 交换**（86移至子节点）
    - 继续检查 → 违反 → **56 与 86 交换**
    - 检查其余分支 → 不违反 → **调整完成**

【大根堆删除堆顶实例】

- **初始状态**：大根堆，堆顶为 **92**
- **删除步骤**：
  - **空出堆顶**：删除92，位置留空
  - **末元素上移**：将 **13** 填入堆顶
  - **调整方向**：**自上而下调整 - 调整过程**：
    - 检查 **上大下小** 性质 → 违反
    - **86 与 13 交换**（13移至子节点）
    - 继续检查 → 违反 → **56 与 13 交换**
    - 检查其余分支 → 不违反 → **调整完成**

【核心要点】

- **删除堆顶通用流程**：**空位 → 末元素补位 → 自上而下调整**
- **调整依据**：小根堆遵循 **上小下大**，大根堆遵循 **上大下小**
- **时间复杂度**：调整过程沿堆高度进行，为 **O(log n)**

4.例题:堆的删除 03:31:27

第五章 第16个考点 数的转换

【考点：堆删除堆顶操作】

- 题目：**大根堆**，删除堆顶元素
- 删除对象：**75**
- 操作步骤：
  - **删除堆顶75**，空出位置
  - **末元素10填入堆顶 - 自上而下调整**：
    - **65与10交换**
    - **10与25交换**
- 调整结果：**65，45，25，65，45，25** 开头
- 答案：**选B**

【考点：树与二叉树的转换】

- 核心结论：
  - **任意一棵树**可转换为**唯一对应的二叉树**
  - **任意一棵二叉树**也可转换为**唯一对应的树**
- 转换关系：**一一对应，双向可逆**

二十、树的转换 **03:32:53**

那我们就看一下这个呃转换的规则是什么？好，那么这样，我们现在这给了一棵树啊，你看这个树分了三个叉嘛，二叉树就不能分三个叉哈，现在我要把它转换成它唯一对应的这个二叉树。我们是这样啊，我们从根结点开始，你看这个根结点是a好，然后他有三个孩子，对不对好？然后呢？这三个孩子互相bcd。呃，因为它都是a的孩子，所以他们是互为兄弟的哈，那么这三个兄弟中呢？我就是把b给他保留，然后后面两个就是c和d就是b的兄弟。他本来是兄弟，但是我现在对他进行降级，我把他从先兄弟降级成他的幼儿子，所以呢，我要做的操作就是把这个变成他的幼儿子接在这里，然后把他又接到这里接到。继续做他的幼儿子啊，有几个兄弟就往下一直做他的幼儿子好，所以我们来改一下，改完之后哎，我们用红色标出来就是原来是兄弟嘛，现在变成了他的幼儿子。c然后呢？后面这个d就做c的幼儿子好d就变成这样了，那么同样啊，我们告诉你一棵二叉树，比如说这就是一棵二叉树，我们也可以把它呃就逆向变回它，变回它的树啊，那么。实际上就是跟刚才这个变的过程反过来啊，转换的过程反过来，那我就是把幼儿儿子变成兄弟，那么这两个就是幼儿子吧，哎，本来就是本来就是相当于说是他的下级嘛，现在给他升级就变成了变成他平级了，就变成兄弟。好，所以把这两个给它转换回转换回他的两个兄弟。c和d好，这个就是二叉树倒树好，我们来看一下这道题啊，这道题它给了一个二叉树，然后把它转换成对应的树，我们来看一下啊。根据我们刚才这个规则，你看啊，这个这个一就是他的幼儿子对不对？好，那我们把它转换成数就要升级嘛，就是本来是他的下级，结果现在跟他平级就给他平级哈，所以实际上这里就是a。然后。b那么这个一就提上来跟他平级就这样好，然后呢？这两个这两个是不是是他的这个幼儿子啊？那就给他提上来变成他的平级好，所以。这里这里那个。这里是d，然后它有两个平急，那就是h和kh和k，然后呢？这两个这个就还是跟着d这个j还是跟着h，所以就直接写在下面就行了啊，注意我们这个时候，因为它只有一个，我们就直接。直接写在下面就可以了，那就是这是g这个是j这就变好了，同样的啊，如果说告诉你右边这个呃，告诉你这个数也可以把它变成这个二叉树，就是我们刚才说的规则就是兄弟变成幼儿子啊。那你看呃，本来这个本来这个e是b的，这个兄弟那就变成了他的变成了他的右边的儿子哈，然后呢，这两个是d的兄弟吧，结果就变成了这里他又儿子，然后这个g和j本来就是跟着他的啊，那就那就。就跟着这两个就完了。

【核心规则：树 ↔ 二叉树转换】

- **树 → 二叉树**：从根结点开始，**保留第一个孩子**，其余兄弟**降级为右孩子**（依次串联）
- **二叉树 → 树**：逆向操作，**右孩子升级为兄弟**

【转换操作示例】

- 树（根a，孩子b、c、d）→ 二叉树：b保留，c作b右孩子，d作c右孩子
- 二叉树 → 树：右孩子链（c、d）**升级为兄弟**

【考点：二叉树 → 树（实例）】

- 结点a、b：b为a左孩子，**结点1（b右孩子）升级为b的兄弟**
- 结点d：h、k为d右孩子链，**升级为d的兄弟**
- 结点g、j：分别保持为h、d的子树

【考点：树 → 二叉树（实例）】

- 兄弟结点e（b的兄弟）→ **降级为b的右孩子**
- 兄弟结点（d的兄弟）→ **降级为d的右孩子**
- 原有子树g、j：保持挂接关系不变

第五章的第17个考点森林的转换，那我们前面讲了树转换成二叉树，那么森林也可以转换成二叉树，森林的转换也与树的转换有相似的这个结论啊，就是每一个森林。哎，每一个森林呢？森林其实就是多棵树的意思啊，每一个森林它也有一个唯一对应的二叉树，同样这个二叉树也可以唯一对应一个森林，那么森林的转换方法里面也要用到树的转换啊。啊，首先森林是多棵树，那我们首先把森林里面的每棵树通过前面树的转换转换成一个二叉树好，那么现在就有。呃，森林里面有几棵树，现在就转换成了若干棵二叉树，然后呢？这若干棵二叉树呃，把第二棵。第二棵这个二叉树就作为它的柚子树，就作为第一棵的柚子树，如果还有第三棵的话，那就第三棵作为第二棵的柚子树就是这样，往后面进行，那么二叉树到森林实际上就是上面这个森林到二叉树，这个过程反过来，那我就先把它的。呃，一个二叉树，先把它的柚子树给它分离好，也就是也就是它有若干个柚子树，把它分离分离完了之后，再把每一个二叉树。那个通过那个我们前面上一个讲的二叉树变成树，那就又还原成了森林，然后下面我们有一个实际的题目，我们直接看这个实际题目操作好，我们看一下这个啊，这就是先首先它就是把森林转换成二叉树，那么这个森林。里面有三棵树，那我们根据我们的方法，我们先把里面的每棵树都转换成它对应的二叉树好，那么规则就是我们前面说的降级嘛，就是兄弟变成幼儿子哈，所以呢，这里就变成了。a，然后。bc第一好，然后这个这个这个是他的兄弟吧哎，那就相当于说把他接在这里啊，好接在这里，这边没有了好，然后这个他没这个鸡，他没有兄弟，但是因为他现在是二叉树，二叉树就是左右，所以你就不要直接写了。下面了，你就把它往这边撇一点，撇一点就相当于说他是这个j就变成他的这个呃左儿子，接下来我们刚才说的他这三棵树都变成二叉树了，之后呢，然后把这个第二棵就作为。作为这个的这个呃柚子树啊，所以他就也就是说相当于说把这个给它接在这个地方来啊，接在这个地方来好，所以就接在这里，我用红色标出来。好，然后呢？再继续把它接上来，那么就是。一。这个f。g好，然后接下来第三棵树就作为第二棵的柚子树，也就是呢，把它接到这个地方来，接到这个地方来啊，我用红色标出来。好那么那么，这里就是l，然后j那么把二叉树转换成对应的森林，实际上就是这个过程，反过来啊，也就是把这个这个给它还原回去好那么。怎么还原那么就是先把它的所有的柚子树都给它分离出来啊，那么就也就是说在这里先把它打断掉啊，那这分离出一个柚子树，然后这里又打断掉，所以这又分离出一个柚子树，所以它实际上就。分离出来，这分离出来这三个啊，然后这三个呢，然后根据我们前面讲的这个二叉树转换成树，每一个对应的转换回去，它转换回它，它转换回它，然后它转换回它，再看一下这个概念题啊，我们说的二叉树与。数与二叉树哎，然后森林与二叉树。

【考情与地位】

- 定位：**第五章第17考点**
- 核心：**森林 ↔ 二叉树转换**（与树转换结论相似）

【核心规则：森林 → 二叉树】

- 森林 = **多棵树**，存在**唯一对应二叉树**
- 步骤：
  - **每棵树**先按“树 → 二叉树”规则转换
  - **第2棵**二叉树作为**第1棵的右子树 - 第3棵**作为**第2棵的右子树**，依次串联

【核心规则：二叉树 → 森林】

- 逆向操作：
  - **分离所有右子树**（打断连接）
  - 每棵分离出的二叉树按“二叉树 → 树”规则还原

【考点：森林 → 二叉树（实例）】

- 森林含3棵树，分别转换为二叉树：
  - 第1棵：a为根，b、c、d按**兄弟降级**规则连接
  - 第2棵：e为根，f、g按规则转换
  - 第3棵：i为根，j为**左孩子**（无兄弟时左挂）
- 合并：**第2棵作为第1棵右子树**，**第3棵作为第2棵右子树**

【考点：二叉树 → 森林（实例）】

- 逆向：**打断所有右子树连接**，分离出3棵二叉树
- 每棵再按“二叉树 → 树”规则还原为原森林中的树

【概念辨析】

- **树 ↔ 二叉树**：唯一对应
- **森林 ↔ 二叉树**：唯一对应

1.例题:二叉树对应的树或森林 **03:39:02**

二叉树与森林转换考点解析

【考点：二叉树与森林转换唯一性】

- 结论：**二叉树与森林转换结果唯一**
- 依据：**转换规则确定，无歧义**
- 题型：**选择题**（判断唯一性）

【考点：森林转二叉树右子树节点数】

- 题干：森林含**三棵树**，节点数分别为 **n1、n2、n3**
- 转换规则：
  - 每棵树先转**对应二叉树**，其**右子树均为空**
  - 后续树作为**前一棵树的右子树**依次接入
- 根结点右子树构成：**第二棵、第三棵树的全部节点**
- 答案：**右子树节点数 = n2 + n3**

二十一、树的遍历 **03:39:56**

第五章的第18个考点数的遍历

【考点定位】

- 主题：**树的遍历**
- 关联：与**二叉树遍历**有相似处，但存在**关键区别**

【核心差异：树无中序遍历】

- 二叉树遍历：**先序、中序、后序、层序**（四种）
- 树的遍历：**无中序**
- 原因：树的分叉可为**多叉**（如三叉）
  - 二叉树分两叉：**有左有右**，故可定义“中”
  - 多叉树：**无法区分左右**，故**无中序**

【遍历结论（必记）】

- 结论一：**树的先序序列 = 其唯一对应二叉树的先序序列**
- 结论二：**树的后序序列 = 其唯一对应二叉树的中序序列**
- 记忆方式：**直接记忆结论，无需深究推导**

【实例说明】

- 给定树结构：根节点 **A**，第二层 **B、C、D**
- 对应二叉树生成：**兄弟节点降级**（转为右子树）
- 验证：该二叉树的**先序**与树的**先序相同**；该二叉树的**中序**与树的**后序相同**

1.例题:树的遍历对应二叉树 03:41:46

我们来看题目啊。这个数我们说数没有中序啊，那就那么你就看把这个中序给它排除掉，好中序没有好，他只有先序后序和层次啊，所以这个选c再看这两个题啊，这两个题表达意思是一样的，这也是我们说的这个结论。好，也就是树和它的树的呃，先序和对应二叉树的先序是相同的，然后树的。这个后序和对应二叉树的中序是相同的，所以这里这个选中序好，这里也是选中序

【考点：树的遍历与二叉树遍历对应关系】

- **树无中序遍历**：仅存在**先序、后序、层次遍历**
- 题目结论：**树与对应二叉树遍历序列关系**
  - **树的先序 = 对应二叉树的先序**
  - **树的后序 = 对应二叉树的中序**
- 解题要点：排除中序，依据对应关系选择
  - 第1题：选 **C**
  - 第2题：选 **中序**
  - 第3题：选 **中序**

二十二、森林的遍历 **03:42:14**

森林的遍历

【考点定位】

- 章节：**第五章第19个考点**
- 考情：**森林遍历基本不考**，核心考查**二叉树遍历**，此处仅需了解

【森林遍历规则】

- 研究对象：**先序森林、后序森林**
- 先序森林规则：**依次先序遍历森林中的每一棵树**
- 后序森林规则：**依次后序遍历森林中的每一棵树**
- 与**树的遍历规则**高度相似，可类比记忆

1.例题:森林的遍历 03:42:44

森林遍历例题讲解

【题目背景】

- 给定**三棵树**组成的森林
- 目的：**练习森林的先序与后序遍历**（非构造森林便利）

【森林先序遍历】

- 规则：**依次对每棵树进行先序遍历**
- 第一棵树：**a → bcd**
- 第二棵树：**e → fg**
- 第三棵树：**i → j**
- 结果序列：**a b c d e f g i j**

【森林后序遍历】

- 规则：**依次对每棵树进行后序遍历**

- 第一棵树：**bcd → a**
- 第二棵树：**fg → e**
- 第三棵树：**j → i**
- 结果序列：**b c d a f g e j i**

【核心结论】

- **森林先序 = 对应二叉树的先序**
- **森林后序 = 对应二叉树的中序**
- 与**树的遍历结论**完全对应

二十三、带权路径长度 03:44:02

第五章的第20个考点带权路径长度

1.路径长度与权值的概念 03:44:15

第一个概念叫做路径长度

【路径长度】

- **定义**：**节点到根节点所经过的边（段）数**
- 示例（以A为根）：
  - **D**：A→···→D，经过2段 → **路径长度为2**
  - **E**：A→···→E，经过2段 → **路径长度为2**
  - **C**：A→C，经过1段 → **路径长度为1**

【权（权值）】

- **定义**：**为节点赋予的数值**
- 示例：D赋值为5 → **权值为5**；E赋值为6 → **权值为6**

2.叶结点的带权路径长度 03:44:59

叶节点的带权路径长度

【核心概念】

- **叶节点带权路径长度**：仅针对**叶节点**（非内部节点）
- 定义：**路径长度 × 该节点权值**
- 示例（二叉树）：
  - 节点D：权值5，路径长度2 → **5×2=10**
  - 节点E：权值6，路径长度2 → **6×2=12**
  - 节点F：权值4，路径长度1 → **4×1=4**
- 要点：**路径长度 = 从根到该节点经过的边数**（段数）

3.二叉树的带权路径长度（WPL） 03:45:52

二叉树带权路径长度（WPL）定义与计算

【核心概念】

- **带权路径长度（WPL）**：二叉树中**所有叶结点**的带权路径长度之和
- 缩写：**WPL**
- 单叶结点带权路径长度 = **权值 × 路径长度**（根到该叶的边数）

【计算步骤】 - 找出**全部叶结点 -** 分别计算每个叶结点的带权路径长度

- 求和得 **WPL**

【示例计算】

- 给定三个叶结点，其带权路径长度分别为：**10、12、4**
- 求和：10 + 12 + 4 = **26**
- 结论：该二叉树 **WPL = 26**

4.例题1:不同形态二叉树的WPL比较 03:46:20

第五章的第21个考点，咱们提出这样一个问题，我们我把n个带全的节点作为叶节点，我可以构造出若干个不同的二叉树啊，比如说我这个n取三的时候。你看我给的这个例子啊，比如说ABC就是三个代权的节点，我把它作为叶节点，我可以构造出，比如说。呃，构造出好多棵不同的形态的二叉树，这里我给了两棵。当然，它不止不止两棵啊，意思就是说就是它可以构造出不同的，那么呢？每一棵二叉树，我以这三个叶节点去计算它的这个wpr，就是我们说的代理权，路径，长度是可以算出一个值来吧，也就是说。呃，左边这棵树可以算出一个值来了，比如说wpl 1啊，另右边这个可以算出来wpl 2，那么算出来他们他这个值算出来之自然就有一个大小。当然，我们刚才说了，它还有呃其他的形态啊，那就是它的这若干种形态算出来的，若干个wpl里面哪个是最小的？我怎样去找这个最小的wpl？那么为了解决这个问题，

【问题引入】

- **核心问题**：n个带权节点作为叶节点，可构造**若干形态不同**的二叉树
- 示例：n=3（节点A、B、C），存在**多棵不同形态**二叉树（不止两棵）
- **WPL计算**：每棵二叉树均可依叶节点计算**带权路径长度（WPL）**
  - 左树 → **WPL_1**
  - 右树 → **WPL_2**
  - 各WPL值存在**大小差异**
- **求解目标**：在**所有形态**对应的**若干WPL值**中，**找出最小值**（即最优二叉树）

二十四、哈夫曼树 **03:47:24**

哈夫曼树的概念与构造方法

【核心概念】

- **哈夫曼树**：为解决**最小WPL（带权路径长度）**问题而提出
- 核心结论：**哈夫曼树的WPL最小**

【构造方法】

- 核心原则：**不断将权值最小的两个结点合并**
- 示例（权值A=1，B=2，C=5）：
  - 第一步：合并A（1）与B（2）→ 新结点权值**3**
  - 第二步：合并新结点（3）与C（5）→ 根结点权值**8**
  - 结果：构造出哈夫曼树

【WPL计算示例】

- A（权值1）：路径长度**2** → 贡献 1×2 = 2
- B（权值2）：路径长度**2** → 贡献 2×2 = 4
- C（权值5）：路径长度**1** → 贡献 5×1 = 5
- **总WPL = 2 + 4 + 5 = 11**
- 结论：**任何其他构造方式的WPL均不小于此值**

【重要结论】

- **n个叶结点的哈夫曼树，总结点数为 2n - 1**
- 推导逻辑：
  - 每次合并**新增1个结点**
  - n个结点需合并**n-1次**
  - 总节点数 = 原n个 + 新增(n-1)个 = **2n - 1**

1.例题:哈夫曼树定义 03:50:07

哈夫曼树概念辨析

【核心考点】哈夫曼树定义与性质

- **WPL定义**：**叶结点权值 × 路径长度（边数）之和**最小

- **A选项错误**：非“叶结点权值之和最小”，需**乘以路径长度**

【选项辨析】

- **B选项正确**：**WPL最小**即为哈夫曼树
- **C选项错误**：**哈夫曼树不唯一**
  - 原因1：**WPL值相同**时，多棵树均可为哈夫曼树
  - 原因2：**叶结点左右交换**不影响路径长度，故WPL不变，形成不同树
- **D选项错误**：非“高度最低”，核心标准为**WPL最小**

2.例题:哈夫曼树计算 03:51:16

铁好再看这道题啊，他说一三五七四个构造哈夫曼树

【考点：哈夫曼树构造与WPL计算】

- **构造规则**：**不唯一**；习惯性**左小右大**
- **构造过程**：
  - 最小两叶合并：**1+3=4** - 当前集合（4,5,7），最小合并：**4+5=9** - 当前集合（7,9），最小合并：**7+9=16**
- **WPL计算**：
  - 7：路径长度1 → **7×1**
  - 5：路径长度2 → **5×2**
  - 1、3：路径长度3 → **(1+3)×3**
  - 总和：**7+10+12=29**

再看一下这两个题啊，这两个题可以一起看，因为他用的就是我们最后那个结论就是n个叶节点构造的哈夫曼数一共有多少个节点啊

【考点：哈夫曼树节点总数结论】

- **核心结论**：**n个叶节点**构造的哈夫曼树，**总节点数 = 2n - 1**
- 推导依据：哈夫曼树为**严格二叉树**（度仅为0或2），**无度为1的节点**
- 应用场景：已知叶节点数，直接求总节点数；或已知总节点数反推叶节点数

3.例题:哈夫曼树分支结点 03:52:47

哈夫曼树节点数量计算

【核心考点】

- **哈夫曼树节点总数**：**2n - 1**（n为叶节点数）
  - 代入n=5：**2×5 - 1 = 9**个节点
- **分支节点数**：**总节点数 - 叶节点数**
  - 计算：**9 - 5 = 4**个分支节点
- **关键结论**：**除叶节点外，其余均为分支节点**

4.例题:哈夫曼树结点总数 03:53:00

哈夫曼树与哈夫曼编码

【考点：哈夫曼树构造结论】

- **直接考查结论**：n个叶子结点构造哈夫曼树
- **总结点数**：**2n - 1**
- 对应选项：**C**

【考点：哈夫曼编码】

- 定位：**第五章第22个考点**
- 核心内容：**哈夫曼编码**（基于哈夫曼树的前缀编码）

二十五、哈夫曼编码 **03:53:12**

哈夫曼编码

【核心考点】哈夫曼编码

- **定义**：借助**哈夫曼树**对若干字符进行编码的方法
- **输入形式**：字符集（如abcd）+ **权值**（或频率等）
- **构造步骤**：
  - 字符→**叶结点**，权值→构造**哈夫曼树 -** 路径标记：**左分支 = 0**，**右分支 = 1 -** 编码获取：**根结点** → **叶结点**路径上的0/1序列

【示例演示】

- 字符：a、b、c；权值：1、2、5
- 构造哈夫曼树：1+2=3，3+5=8
- 编码结果：
  - **a**：路径00→ **编码00**
  - **b**：路径01→ **编码01**
  - **c**：路径1→ **编码1**

1.例题:哈夫曼编码 03:55:07

哈夫曼编码构造实例

【核心考点】哈夫曼编码构造流程

- **初始条件**：6个字符 = 6个叶节点，频率 = 权值
- **构造原则**：每次选取**权值最小的两个节点**合并
  - 习惯性：**小者居左，大者居右**（非强制）

【构造步骤】 - 首轮合并： - 5 + - 8 = - **3 - 次轮合并**： - 3 + - 4 = - **7 - 第三轮合并**： - 2 + - 3 = - **5 - 第四轮合并**： - 7 + - 8 = - **5 - 末轮合并**： - 5 + - 5 = - **0**（根节点）

【编码规则】

- **左分支**：标记 **0**
- **右分支**：标记 **1**
- 路径：**从根节点至叶节点**，依次记录分支标记

【编码结果】

- 字符（权值 - 5）：**1000**
- 字符（权值 - 8）：**1001**
- 字符（权值 - 4）：**1011**
- 字符（权值 - 2）：**00**
- 字符（权值 - 3）：**01**
- 字符（权值 - 8）：**11**